Thiago Fernandes Pinheiro de Medeiros Rodrigues

Navegação Autônoma Local: Estudo Empírico sobre o Ajuste de Modelos de Linguagem-Visão Compactos para o Uso de Interfaces Gráficas

Natal – RN

Julho de 2026

Thiago Fernandes Pinheiro de Medeiros Rodrigues

Navegação Autônoma Local: Estudo Empírico sobre o Ajuste de Modelos de Linguagem-Visão Compactos para o Uso de Interfaces Gráficas

Trabalho de Conclusão de Curso de Engenharia de Computação da Universidade Federal do Rio Grande do Norte, apresentado como requisito parcial para a obtenção do grau de Bacharel em Engenharia de Computação

Orientador: Luiz Affonso Henderson Guedes de Oliveira

Coorientador: Carlos Augusto Prolo

Universidade Federal do Rio Grande do Norte – UFRN

Departamento de Engenharia de Computação e Automação – DCA

Curso de Engenharia de Computação

Natal – RN

Julho de 2026

Thiago Fernandes Pinheiro de Medeiros Rodrigues

Navegação Autônoma Local: Estudo Empírico sobre o Ajuste de Modelos de Linguagem-Visão Compactos para o Uso de Interfaces Gráficas

Trabalho de Conclusão de Curso de Engenharia de Computação da Universidade Federal do Rio Grande do Norte, apresentado como requisito parcial para a obtenção do grau de Bacharel em Engenharia de Computação

Orientador: Luiz Affonso Henderson Guedes de Oliveira

Coorientador: Carlos Augusto Prolo

Trabalho aprovado. Natal – RN, 02 de Julho de 2026:

Prof. Dr. Luiz Affonso Henderson Guedes de Oliveira - Orientador
UFRN

Prof. Dr. Carlos Augusto Prolo - Coorientador
UFRN

Dr. Breno Santana Santos
UFRN

Natal – RN
Julho de 2026

Dedico este trabalho aos meus pais e avós, pelo amor incondicional, pelo apoio constante e por serem a base de tudo que sou.

AGRADECIMENTOS

Primeiramente, dedico meus mais sinceros agradecimentos à minha família, aos meus amigos e a todas as pessoas que, direta ou indiretamente, me inspiraram e me apoiaram incondicionalmente ao longo de toda essa jornada. Sem o suporte emocional e o incentivo de vocês, a conclusão desta etapa não seria possível.

Expresso minha profunda gratidão aos meus orientadores, Prof. Luiz Affonso e Prof. Carlos Augusto Prolo, pela paciência, dedicação e por me guiarem com excelência e parceria na construção deste trabalho. O apoio de vocês foi imprescindível para o desenvolvimento desta pesquisa.

Agradeço imensamente ao Dr. Breno Santana Santos por aceitar o convite para compor a banca examinadora. Suas correções, críticas construtivas e avaliação minuciosa foram fundamentais para o refinamento e a garantia da qualidade final deste documento.

Sou muito grato ao Departamento de Engenharia de Computação e Automação (DCA) da UFRN e a todo o seu corpo docente pelo vasto aprendizado proporcionado ao longo destes anos. Agradeço, de forma especial, ao laboratório Model2AI do departamento, pela disponibilização da infraestrutura e dos recursos computacionais que foram essenciais para os experimentos deste projeto.

Estendo meus agradecimentos a todos os professores e mentores que cruzaram meu caminho. Gostaria de registrar também minha imensa gratidão à equipe da DtLabs, não apenas por ser um ambiente onde aprendo continuamente com meus colegas de profissão, mas especialmente pelo apoio e pela flexibilidade de horários concedidos na reta final de elaboração deste trabalho.

Por fim, agradeço aos meus colegas de curso. O excelente ambiente que construímos juntos, as trocas de conhecimento e as experiências compartilhadas tornaram os desafios da graduação muito mais leves e gratificantes. Muito obrigado a todos.

"No meio da dificuldade
encontra-se a oportunidade."
(Albert Einstein)

RESUMO

A automação autônoma de interfaces gráficas de usuário (GUIs) por agentes de uso de computador (CUAs, do inglês *Computer Use Agents*) é dominada por modelos de grande porte, cuja dependência de infraestrutura de larga escala impõe barreiras de custo, latência e privacidade. Essas barreiras motivam o interesse por agentes compactos, executáveis em *hardware* de consumo, mas as técnicas de treinamento adequadas a essa faixa de escala e seus principais gargalos permanecem pouco caracterizados. Diante disso, este trabalho investiga empiricamente se o ajuste fino por aprendizado por reforço melhora o desempenho de um CUA compacto, na faixa de quatro bilhões de parâmetros, treinado exclusivamente sob restrições de *hardware* comercial. Para tanto, ajusta-se um modelo de visão e linguagem (VLM) da família Qwen3-VL sobre o conjunto de dados AgentNet, por meio de aprendizado por reforço (RL), adotando o algoritmo de otimização de política de sequência relativa ao grupo (GSPO) sob regime *off-policy*, com episódios de passo único e uma função de recompensa multicritério. O modelo resultante é avaliado nos *benchmarks ScreenSpot-Pro* e *OSWorld-Verified*, em comparação controlada com a rede base equivalente. Os resultados revelam um ganho consistente em ancoramento espacial (*grounding*), com a acurácia geral evoluindo de 33,7% para 38,7%, concentrado na localização de elementos textuais. Em contraste, observa-se uma leve regressão no desempenho agêntico de horizonte longo, de 15,1% para 11,3%, atribuída predominantemente a uma falha de sub-terminação, e não à perda de capacidade perceptiva ou de manipulação da interface. A análise das métricas internas associa esse comportamento à escassa representação da ação de encerramento e à curta duração do treino, indícios reforçados por um treino preliminar que demonstra a viabilidade de uma política de terminação calibrada quando essa ação recebe representação suficiente. Conclui-se que a estratégia de alinhamento por reforço é estruturalmente viável para arquiteturas compactas e amplamente suscetível a otimizações, oferecendo um referencial empírico documentado para investigações futuras sobre CUAs locais.

Palavras-chaves: Agentes de uso de computador; Aprendizado por reforço; Modelos multimodais; Modelos pequenos; Interface gŕafica de usuário.

ABSTRACT

The autonomous automation of graphical user interfaces (GUIs) by computer use agents (CUAs) is dominated by large models, whose dependence on large-scale infrastructure imposes cost, latency, and privacy barriers. These barriers motivate interest in compact agents executable on consumer hardware, but the appropriate training techniques for this scale range and their main bottlenecks remain poorly characterized. Given this, this work empirically investigates whether reinforcement learning fine-tuning improves the performance of a compact CUA, in the four-billion-parameter range, trained exclusively under commercial hardware constraints. To this end, a vision-language model (VLM) from the Qwen3-VL family is fine-tuned on the AgentNet dataset through reinforcement learning (RL), adopting the Group Relative Sequence Policy Optimization (GSPO) algorithm under an off-policy regime, utilizing single-step episodes and a multi-criteria reward function. The resulting model is evaluated on the ScreenSpot-Pro and OSWorld-Verified benchmarks in a controlled comparison with the equivalent base network. The results reveal a consistent gain in spatial grounding, with overall accuracy improving from 33.7% to 38.7%, primarily concentrated in the localization of textual elements. In contrast, a slight regression in long-horizon agentic performance is observed, dropping from 15.1% to 11.3%, predominantly attributed to an under-termination failure rather than a loss of perceptual or interface manipulation capabilities. An analysis of internal metrics associates this behavior with the scarce representation of the termination action and the short training duration, indications reinforced by a preliminary training phase that demonstrates the feasibility of a calibrated termination policy when this action receives sufficient representation. It is concluded that the reinforcement alignment strategy is structurally viable for compact architectures and highly amenable to optimization, providing a documented empirical baseline for future investigations into local CUAs.

Keywords: Computer use agents; Reinforcement learning; Vision-language models, Small models; Graphic user interface.

LISTA DE ILUSTRAÇÕES

Figura 1 – À esquerda, o funcionamento do mecânismo de atenção por produto escalar normalizado. À direita, o funcionamento do modulo de multi-atenção. 25
Figura 2 – Arquitetura original do *Transformer*. 26
Figura 3 – Fluxo de processamento: modelo de linguagem e visão (VLM). 28
Figura 4 – Interação agente-ambiente em um MDP. 30
Figura 5 – Função objetivo substituta clipada do PPO para vantagens positivas e negativas. 35
Figura 6 – Formato estrito exigido pela recompensa de formato. A coluna da esquerda contém os blocos `<observation>` e `<reason>` em linguagem natural, enquanto a coluna da direita apresenta o bloco `<answer>`, formatado como uma lista de dicionários em que cada elemento reúne uma descrição da ação e a respectiva chamada de função. 44
Figura 7 – Organização estrutural do conjunto de dados AgentNet. 47
Figura 8 – Distribuição de frequência das ações válidas no *dataset* AgentNet após a filtragem dos dados (Seção 3.5.1). As cores agrupam as ações em comandos de ponteiro e rolagem, entradas de teclado e controle de fluxo. 50
Figura 9 – Estrutura da entrada fornecida ao modelo durante a inferência. A mensagem do sistema é apresentada em verde, as do assistente em azul e as do usuário em vermelho. A primeira linha representa o histórico até a penúltima etapa, enquanto a segunda linha mostra o passo imediatamente anterior, que mistura mensagens de assistente e usuário. Por fim, inclui-se a captura atual acompanhada do objetivo. 52
Figura 10 – Comparação da acurácia no *ScreenSpot-Pro* entre o modelo base e o treinado. 58
Figura 11 – Comparação de trajetórias em uma tarefa do *GIMP*. Ambos atingem o objetivo com o mesmo prefixo de ações; o modelo treinado prossegue além do ponto de conclusão e invalida o resultado. 60
Figura 12 – Evolução das recompensas no treino principal. O formato satura próximo de seu teto de 0,6 nos primeiros passos. A acurácia, único componente com margem, permanece quase plana. 61
Figura 13 – Recompensa de acurácia em função do número de exemplos processados. O eixo em exemplos atenua a diferença de tamanho de *batch*, mas o treino principal acumulou muito menos atualizações de parâmetros que o preliminar, de modo que a comparação é apenas indicativa. 62

Figura 14 – Comprimento médio das gerações (a) e fração de grupos com desvio padrão de recompensa nulo (b), em função do número de exemplos processados, para os dois treinos. 63

LISTA DE TABELAS

Tabela 1 – Comparação dos trabalhos relacionados segundo os eixos de projeto. . . 40
Tabela 2 – Classificação das variáveis do experimento. 55
Tabela 3 – Matriz experimental: níveis dos fatores em cada configuração. 55
Tabela 4 – Acurácia no *benchmark ScreenSpot-Pro*, por tipo de alvo. 58
Tabela 5 – Taxa de sucesso no *OSWorld-Verified* por domínio (crédito parcial). . . 59
Tabela 6 – Indicadores comportamentais agregados no *OSWorld-Verified*. 60
Tabela 7 – Calibração da ação `terminate` no treino preliminar, por faixa de passos (fase avançada do treino). 64

LISTA DE ABREVIATURAS E SIGLAS

ChatML	*Chat Markup Language*
CUA	*Computer Use Agent*
DOM	*Document Object Model*
DPO	*Direct Preference Optimization*
GAE	*Generalized Advantage Estimation*
GRPO	*Group Relative Policy Optimization*
GSPO	*Group Sequence Policy Optimization*
GUI	*Graphical User Interface*
IA	Inteligência Artificial
JSON	*JavaScript Object Notation*
KL	Kullback-Leibler
LLM	*Large Language Model*
LN	*Layer Normalization*
MDP	*Markov Decision Process*
MLP	*Multilayer Perceptron*
MoE	*Mixture of Experts*
PLN	Processamento de Linguagem Natural
PPO	*Proximal Policy Optimization*
regex	*Regular Expression*
RL	*Reinforcement Learning*
RLHF	*Reinforcement Learning from Human Feedback*
SFT	*Supervised Fine-Tuning*
VC	Visão Computacional

ViT	*Vision Transformer*
VLM	*Vision-Language Model*
VRAM	*Video Random Access Memory*
XML	*Extensible Markup Language*

LISTA DE SÍMBOLOS

A	Conjunto de ações possíveis
A_t	Ação tomada no instante t
A_i	Sequência (resposta) amostrada i
C	Número de canais da imagem
D	Dimensão de projeção dos *embeddings*
D_{KL}	Divergência de Kullback-Leibler
$\mathbf{E}$	Matriz de projeção linear dos *patches*
$\mathbf{E}_{pos}$	*Embeddings* de posição
G	Número de amostras (trajetórias) por grupo
G_t	Retorno acumulado a partir do instante t
$J(\theta)$	Função objetivo a ser otimizada
K	Matriz de chaves (*keys*)
K_g	Comprimento (número de passos) da trajetória g
L	Número de camadas da rede
N	Número de ações previstas pelo modelo
N_p	Número de *patches* da imagem
P	Tamanho da aresta do *patch*
$P(q)$	Distribuição sobre os comandos de entrada
Q	Matriz de consultas (*queries*)
R	Conjunto de recompensas possíveis
R_{passo}	Recompensa total por passo
$R_{formato}$	Recompensa de formato
R_{tipo}	Recompensa de tipo de ação

R_{acerto}	Recompensa de ação completa (acerto de argumentos)
$\mathbf{R}$	Conjunto das recompensas totais das sequências do grupo
R_t	Recompensa recebida no instante t
S	Conjunto de estados possíveis
S_t	Estado no instante t
T	Último instante (etapa terminal) do episódio
V	Matriz de valores (*values*)
a	Ação genérica
$b(s)$	Função de linha de base (*baseline*)
d_k	Dimensão do vetor-chave
g	Índice da trajetória amostrada
h	Número de cabeças de atenção
$\mathbf{k}$	Vetor de chave (*key*)
k	Índice de passo ou somatório
$p(s', r \mid s, a)$	Função de transição do ambiente
$\mathbf{q}$	Vetor de consulta (*query*)
q	Comando ou instrução (*prompt*)
$q_\pi(s, a)$	Função de valor de ação sob a política π
$\mathbf{r}$	Conjunto das recompensas do grupo (passo único)
r	Recompensa genérica
r_g	Recompensa da trajetória g
$\tilde{r}_g$	Recompensa normalizada da trajetória g
$r_t(\theta)$	Razão de amostragem por importância
s	Estado genérico
s'	Estado seguinte
s_0	Estado inicial do episódio

$s_i(\theta)$	Razão de importância da sequência i
t	Instante (passo de tempo)
$\mathbf{v}$	Vetor de valor (*value*)
$v_\pi(s)$	Função de valor de estado sob a política π
$\hat{v}(s, w)$	Estimativa da função de valor (crítico)
w	Pesos do estimador de valor
w_1	Coeficiente de ponderação da recompensa de formato
w_2	Coeficiente de ponderação das recompensas de tarefa
$\mathbf{x}_{\text{classe}}$	*Token* de classe
$\mathbf{x}_p^i$	*Embedding* do i-ésimo *patch*
x, y	Coordenadas espaciais da ação de ponteiro
$\mathbf{y}$	Vetor de saída do ViT
$\mathbf{z}_\ell$	Saída da camada ℓ
α	Fator de aprendizado
β	Coeficiente da penalização por divergência KL
γ	Fator de desconto
ϵ	Hiperparâmetro de corte (*clipping*)
θ	Parâmetros da política
λ	Fator de ponderação da estimativa generalizada da vantagem
$\mu(s)$	Métrica de ocupação dos estados
π	Política de decisão
π_θ	Política sendo otimizada
$\pi_{\theta_{coleta}}$	Política de coleta (referência)
ℓ	Índice da camada da rede
$\mathcal{S}$	Espaço de estados
$\mathcal{A}$	Espaço de ações

$\hat{\mathcal{A}}_t$	Estimativa da vantagem no instante t
$\mathbb{R}$	Conjunto dos números reais
$\mathbb{E}$	Esperança (valor esperado)
∇	Gradiente
Δ	Variação
$\in$	Pertence
$\propto$	Proporcional a
$\doteq$	Igual por definição
$\sim$	Distribuído conforme
$\lvert A_i \rvert$	Comprimento (número de *tokens*) da sequência A_i

SUMÁRIO

1 INTRODUÇÃO 19
1.1 Motivação e justificativa 20
1.2 Objetivos 21
1.3 Metodologia de pesquisa 21
1.4 Organização do trabalho 22

2 REFERENCIAL TEÓRICO 24
2.1 *Modelos de visão e linguagem* 24
2.1.1 *Transformers* 24
2.1.2 *Vision transformers* 26
2.1.3 *Integrando linguagem e visão* 27
2.1.4 *Pré-treinamento e ajuste fino* 29
2.2 *Aprendizado por reforço* 30
2.2.1 *Processo de decisão de Markov* 30
2.2.2 *Otimização de política por gradiente* 32
2.2.3 *Otimização de política proximal* 33
2.2.4 *Otimização de política relativa ao grupo* 35
2.2.5 *Otimização de política de sequência relativa ao grupo* 36
2.3 *Modelos para uso de interface gráfica* 37
2.3.1 *Navegação baseada em frameworks* 37
2.3.2 *Modelos nativos de navegação* 37
2.3.3 *Do ajuste supervisionado ao reforço* 38
2.4 *Trabalhos relacionados* 38
2.4.1 *Recompensas verificáveis no ajuste de agentes de GUI* 39
2.4.2 *Projetos abertos e modelos nativos de desktop* 39
2.4.3 *Posicionamento da proposta* 40

3 METODOLOGIA EXPERIMENTAL 41
3.1 Definição do problema e objetivo do experimento 41
3.2 Variáveis de resposta 41
3.2.1 Respostas finais 42
3.2.2 Respostas de processo 42
3.3 Formulação da recompensa 42
3.3.1 Recompensa de formato ($R_{formato}$) 43
3.3.2 Recompensa de tarefa (R_{tipo} e R_{acerto}) e avaliação sequencial 44

3.3.3 Tolerâncias de validação . . . 45
3.4 Componentes fixos do experimento . . . 45
3.4.1 Modelo base . . . 45
3.4.2 Conjunto de dados . . . 47
3.4.3 Regime de treino e enquadramento temporal . . . 48
3.4.4 Algoritmo de otimização . . . 49
3.4.5 Espaço de ações . . . 50
3.4.6 Estrutura das entradas . . . 51
3.5 Preparação dos dados de treino . . . 52
3.5.1 Filtragem e curadoria das trajetórias . . . 52
3.5.2 Padronização das ações e geração dos rótulos . . . 53
3.6 Configurações experimentais e execução . . . 54
3.7 Plano de análise . . . 56

4 RESULTADOS . . . 57
4.1 Protocolo de avaliação . . . 57
4.2 Desempenho em ancoramento espacial . . . 58
4.3 Desempenho em tarefas agênticas . . . 59
4.3.1 Análise comportamental . . . 59
4.4 Métricas internas de treinamento . . . 60
4.4.1 Saturação do formato e estreiteza do sinal de acurácia . . . 61
4.4.2 Comprimento das gerações e variância das amostras . . . 62
4.5 Dinâmica de terminação . . . 63
4.6 Ameaças à validade . . . 64
4.6.1 Fatores não isolados entre as configurações . . . 64
4.6.2 Execução única sem replicação . . . 65
4.6.3 Variância de medição no OSWorld-Verified . . . 65

5 CONSIDERAÇÕES FINAIS . . . 67
5.1 Dificuldades enfrentadas no desenvolvimento do estudo . . . 67
5.2 Alcance das conclusões e suas limitações . . . 68
5.3 Continuidade da pesquisa e trabalhos futuros . . . 70

REFERÊNCIAS . . . 72

6 APÊNDICE . . . 76
6.1 *Prompt* do sistema do agente . . . 76

1 INTRODUÇÃO

A automação da interação com interfaces gráficas de usuário (GUIs, do inglês *Graphical User Interfaces*) é um objetivo de longa data e de difícil realização. Construídas para o consumo humano, GUIs são ambientes dinâmicos e visualmente densos, nos quais a função de um elemento frequentemente depende de pistas visuais sutis e relações espaciais entre itens, como ícones desprovidos de rótulos textuais ou menus sobrepostos cuja visibilidade altera-se mediante interação. Operá-las de forma autônoma exige mais do que a identificação passiva de componentes. Demanda-se a tradução de objetivos abstratos em sequências de ações coerentes, executadas sobre um espaço de estados imprevisível. Historicamente, tentativas de automação anteriores foram severamente limitadas por essa complexidade, apoiando-se em heurísticas manuais, seletores frágeis e fluxos rigidamente codificados que se quebram diante de qualquer alteração de *layout* ou de cenário, o que restringe seu uso a contextos controlados e previsíveis.

Modelos de linguagem de grande escala (LLMs, do inglês *Large Language Models*) e suas extensões para o domínio multimodal, os modelos de visão e linguagem (VLMs, do inglês *Vision-Language Models*), oferecem um novo caminho para esse problema. As capacidades de raciocínio e percepção exibidas por essas redes permitem abordar a navegação de interfaces pela interpretação direta da tela, de maneira análoga a um operador humano, superando abordagens com base em regras explícitas. Dessa evolução emergem os agentes de uso de computador (CUAs, do inglês *Computer Use Agents*), sistemas que convertem objetivos descritos em linguagem natural em sequências de operações definidas atuando diretamente sobre as coordenadas dos *pixels*. Por reunir percepção visual, raciocínio e ancoramento espacial preciso (*grounding*), a navegação autônoma de GUIs firma-se como um marco relevante para a agência autônoma, com paralelos diretos em domínios como a robótica e a automação de processos, onde o modelo deve mediar a conversão de intenções em ações úteis dentro de um ambiente não estruturado.

As primeiras abordagens fundamentadas nesse paradigma apoiavam-se em *frameworks* de orquestração, nos quais um LLM de propósito geral recebia representações textuais da tela, como árvores de acessibilidade ou o *Document Object Model* (DOM), e operava o ambiente por meio de regras e ferramentas codificadas manualmente (ZHOU et al., 2024; DENG et al., 2023). Embora mais flexíveis que sistemas puramente baseados em regras, tais soluções herdam parte de sua fragilidade e dependem de metadados nem sempre disponíveis ou corretamente implementados pelos desenvolvedores de *software*. A consolidação dos VLMs permitiu a transição para modelos nativos de navegação, que interpretam a interface diretamente a partir de sua representação visual e tratam a navegação

como um problema unificado de percepção e ação (XU et al., 2024; QIN et al., 2025). Mais recentemente, acompanhando o êxito do aprendizado por reforço (RL, do inglês *Reinforcement Learning*) na indução de raciocínio em LLMs (DeepSeek-AI, 2025), métodos baseados em reforço passaram a ser aplicados ao treinamento desses agentes, dispensando a rotulação humana exaustiva do ajuste supervisionado e incentivando a emergência de estratégias próprias de navegação ao premiar a corretude do resultado final em vez da imitação estrita de trajetórias previamente mapeadas (LU et al., 2025b; LUO et al., 2025).

Apesar dos progressos, o estado da arte em CUAs é dominado por modelos de grande porte (WANG et al., 2025; WANG et al., 2025), frequentemente proprietários e dependentes de infraestrutura computacional de larga escala. Essa configuração impõe barreiras de custo, latência e privacidade que restringem a adoção desses agentes e motivam o interesse por alternativas compactas, capazes de executar localmente em *hardware* de consumo. Aproximar tais modelos do desempenho dos sistemas de fronteira configura uma frente de pesquisa ampla e ativa, na qual permanecem em aberto questões sobre as técnicas de treinamento favoráveis a essa faixa de escala, seus principais gargalos e quais deles podem ser contornados sob recursos restritos.

1.1 Motivação e justificativa

A predominância de modelos de fronteira na operação autônoma de interfaces gráficas concentra três limitações que restringem sua aplicação prática. A primeira é o custo de inferência, uma vez que modelos proprietários de larga escala cobram por uso e tornam onerosa a automação contínua ou em volume. A segunda é a latência, agravada pela natureza sequencial da tarefa: a conclusão de um único objetivo exige dezenas de predições encadeadas, e a dependência de processamento remoto acumula o atraso de rede a cada passo. A terceira é a privacidade, visto que a operação de um CUA requer o envio contínuo de capturas de tela, potencialmente sensíveis, a servidores de terceiros, condição incompatível com cenários corporativos, jurídicos ou clínicos nos quais o conteúdo exibido na tela é confidencial.

Agentes compactos capazes de executar localmente em *hardware* de consumo respondem diretamente a essas limitações. A inferência local elimina a latência de rede e o custo por chamada, além de manter as capturas de tela no próprio dispositivo, preservando a confidencialidade dos dados. A operação sem dependência de conexão contínua viabiliza o uso em ambientes de conectividade restrita ou em dispositivos de borda, e a redução da barreira de infraestrutura amplia o acesso à pesquisa e ao desenvolvimento desses sistemas. Sob outra perspectiva, o regime de poucos parâmetros encontra sua maior aptidão em domínios estreitos e bem delimitados, nos quais a capacidade do modelo pode concentrar-se sobre uma distribuição restrita de estados e ações, o que abre espaço para aplicações

especializadas em fluxos de trabalho ou famílias de aplicações específicas.

Aproximar esses agentes do desempenho dos sistemas de fronteira permanece, contudo, um problema em aberto. As técnicas de treinamento mais adequadas a essa faixa de escala, os gargalos que a limitam e as estratégias capazes de contorná-los sob recursos restritos ainda carecem de caracterização empírica. Variantes recentes de aprendizado por reforço que dispensam um modelo crítico, como a otimização de política relativa ao grupo (GRPO, do inglês *Group Relative Policy Optimization*) (SHAO et al., 2024) e à sequência (GSPO, do inglês *Group Sequence Policy Optimization*) (ZHENG et al., 2025), reduzem substancialmente o custo de memória do ajuste ao eliminar a necessidade de manter cópias adicionais da rede em memória de vídeo (VRAM, do inglês *Video Random Access Memory*). Essa eficiência torna factível investigar o alinhamento de modelos compactos fora de ambientes de larga escala, condição que fundamenta a viabilidade deste estudo e justifica o exame empírico de seus limites e capacidades sob *hardware* comercial.

1.2 Objetivos

O objetivo geral deste trabalho é caracterizar empiricamente as capacidades e as limitações de um agente de uso de computador compacto, na faixa de quatro bilhões (4B) de parâmetros, ajustado por aprendizado por reforço exclusivamente sob restrições de *hardware* comercial. Busca-se identificar quais competências de navegação emergem do procedimento de ajuste, quais permanecem deficitárias e que fatores estruturais do treino condicionam o desempenho resultante, produzindo um referencial empírico documentado sobre o alinhamento de modelos dessa escala para a operação autônoma de interfaces gráficas.

Para alcançar esse objetivo, busca-se: (i) ajustar um modelo compacto de aproximadamente quatro bilhões de parâmetros, combinando técnicas recentes de aprendizado por reforço compatíveis com *hardware* comercial, (ii) avaliar o modelo resultante sobre *benchmarks* consolidados de interação com interfaces, quantificando seu desempenho na execução de tarefas completas e na precisão de ancoramento espacial, e (iii) diagnosticar os gargalos do procedimento de treino, relacionando as capacidades e vulnerabilidades observadas às características estruturais do ajuste e apontando direções para investigações posteriores.

1.3 Metodologia de pesquisa

Quanto à natureza, esta pesquisa classifica-se como aplicada, pois produz um artefato concreto, um agente ajustado, e conhecimento prático sobre seu treinamento, em vez de teoria de propósito geral. Quanto aos objetivos, assume caráter exploratório e

descritivo. Explora uma região pouco ocupada do espaço de projeto de agentes de GUI, definida pela combinação de modelo compacto, domínio de *desktop*, regime *off-policy* e otimização por GSPO, e descreve o comportamento do modelo resultante sem a pretensão de estabelecer relações causais entre as múltiplas escolhas de configuração. Quanto à abordagem, é predominantemente quantitativa, apoiada em métricas de *benchmark* e em grandezas internas de treinamento, complementada por uma interpretação qualitativa dos modos de falha observados nas trajetórias do agente.

Quanto aos procedimentos técnicos, o trabalho organiza-se como um estudo de caso único de caráter experimental, centrado na caracterização aprofundada de um único agente compacto em seu contexto de avaliação. O núcleo probatório desse estudo é uma comparação experimental controlada, na qual o modelo ajustado e a rede base equivalente são avaliados sob condições idênticas, de modo a isolar o efeito do ajuste sobre o desempenho. O planejamento segue os princípios de definição de fatores, variáveis de resposta e componentes fixos discutidos por Antony (2023), adaptados ao contexto computacional. Cabe registrar que o estudo não constitui um delineamento fatorial replicado. As duas configurações de treino executadas diferem em múltiplos fatores simultâneos e carecem de replicação, de modo que a inferência estatística de causa e efeito entre configurações, discutida na Seção 4.6, permanece fora de seu alcance.

A condução do estudo seguiu uma sequência definida de etapas. Inicialmente, levantaram-se as abordagens contemporâneas de ajuste de agentes de GUI, a partir das quais se fixaram o modelo base e o conjunto de dados. Em seguida, formularam-se a função de recompensa multicritério e o processo de curadoria das trajetórias, que convertem as demonstrações brutas em amostras de treino. O ajuste foi então executado em duas configurações, uma preliminar e exploratória em placa única de consumo e outra principal em infraestrutura de servidor alugada, ambas sob parâmetros compatíveis com *hardware* comercial. Por fim, o modelo resultante foi avaliado em dois *benchmarks* consolidados e o procedimento de treino foi analisado a partir de suas métricas internas, com vistas ao diagnóstico de gargalos e à formulação de direções de melhoria.

1.4 Organização do trabalho

Além desta introdução, este trabalho organiza-se em quatro capítulos. O Capítulo 2 apresenta o referencial teórico, cobrindo os fundamentos dos modelos de visão e linguagem, do aprendizado por reforço e dos modelos nativos voltados ao uso de interfaces gráficas, além dos trabalhos relacionados que situam a proposta. O Capítulo 3 detalha a metodologia experimental, incluindo a definição do problema, as variáveis de resposta, a formulação da recompensa, a preparação dos dados e as configurações de treino executadas. O Capítulo 4 reporta e discute os resultados, tanto sob a ótica das métricas internas de convergência do

treinamento quanto do desempenho final nos *benchmarks*, acompanhados da análise de ameaças à validade. Por fim, o Capítulo 5 reúne as considerações finais, as dificuldades e limitações do estudo e as direções para trabalhos futuros.

2 REFERENCIAL TEÓRICO

Este capítulo reúne os fundamentos teóricos que sustentam a proposta deste trabalho. Parte-se dos modelos de visão e linguagem, que fornecem a base perceptiva e cognitiva dos agentes, passando pelo aprendizado por reforço, paradigma empregado no ajuste fino, até chegar aos modelos voltados ao uso de interfaces gráficas. Ao final, revisam-se os trabalhos relacionados que definem o contexto imediato da pesquisa.

2.1 *Modelos de visão e linguagem*

Os modelos de visão e linguagem (VLMs) resultam da convergência entre o processamento de linguagem natural e a visão computacional em uma única arquitetura capaz de raciocinar sobre texto e imagem de forma conjunta. Esta seção percorre os elementos que os viabilizam, começando pelo *transformer* e sua adaptação ao domínio visual, seguindo pela integração das duas modalidades e pelos regimes de treinamento que consolidam suas capacidades.

2.1.1 *Transformers*

O *transformer* Vaswani et al. (2017) é a peça central por trás dos avanços em Processamento de Linguagem Natural (PLN) feitos na última decada. Arquiteturas baseadas em seu uso tornaram-se a abordagem principal para tarefas de linguagem em geral, englobando desde aplicações em tradução a raciocínio lógico.

O algoritmo provém diversas vantagens em relação às abordagens anteriores devido ao seu foco único no uso de mecanismos de atenção. Dentre essas, destacam-se seu grande potencial para escalabilidade da dados e a maior facilidade no aprendizado de dependências de longa distância, sejam essas semânticas ou estruturais, que torna-se possível devido à presença de caminhos curtos entre os pares de palavras em um dado texto.

O mecânismo de atenção é relativamente simples, sendo projetado para operar em cima de representações vetoriais de palavras, denominadas *embeddings*. Ao receber uma entrada, atribui-se a cada termo, ou *token*, três vetores distintos: $\mathbf{q}$, $\mathbf{k}$ e $\mathbf{v}$. Cada um é computado através de uma projeção linear própria aprendida pelo modelo. O *embedding de consulta* ($\mathbf{q}$) de cada item é comparado com os *vetores de chave* ($\mathbf{k}$) do texto através de uma função de similaridade. Vaswani et al. (2017) a define como o produto escalar dos vetores normalizado pela raiz da dimensão d_k do vetor-chave. O resultado é alimentado a uma camada de *softmax* que atribui pesos às relações entre o termo de interesse e as

palavras ao seu redor, incluindo a si mesmo.

$$\text{Atenção}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2.1}$$

Por fim, os *vetores de valor* ($\mathbf{v}$) de cada *token* são somados de forma ponderada de acordo com importâncias estabelecidas na etapa anterior. Esse processo é paralelizado agregando os três tipos de representação em matrizes e ajustando o cálculo da similaridade, obtendo-se uma matriz final de pesos, como visto na Equação 2.1. Cada linha pode ser somada de forma a obter os *embeddings* atualizados de cada item.

Para cada bloco de atenção, o modelo atualiza as representações dos termos consultados com base em suas relações com os vetores-chave ao seu redor. Visto que as projeções dos *embeddings* em cada camada são aprendidas, isso torna o modelo capaz de capturar uma variedade de conceitos semânticos ou sintáticos pelo uso de espaços vetoriais distintos: um pode capturar o conceito de temperatura, enquanto outro captura assertividade. A fim de maximizar esse efeito, o *transformer* usa modulos de multi-atenção (Figura 1), onde h projeções distintas e cálculos de atenção são computados em paralelo e seus resultados concatenados e alimentados á uma camada linear. Para permitir o aprendizado de ideias atreladas à ordenação, a rede também inclui o cálculo de vetores de posição, que são somados à entrada no inicio.

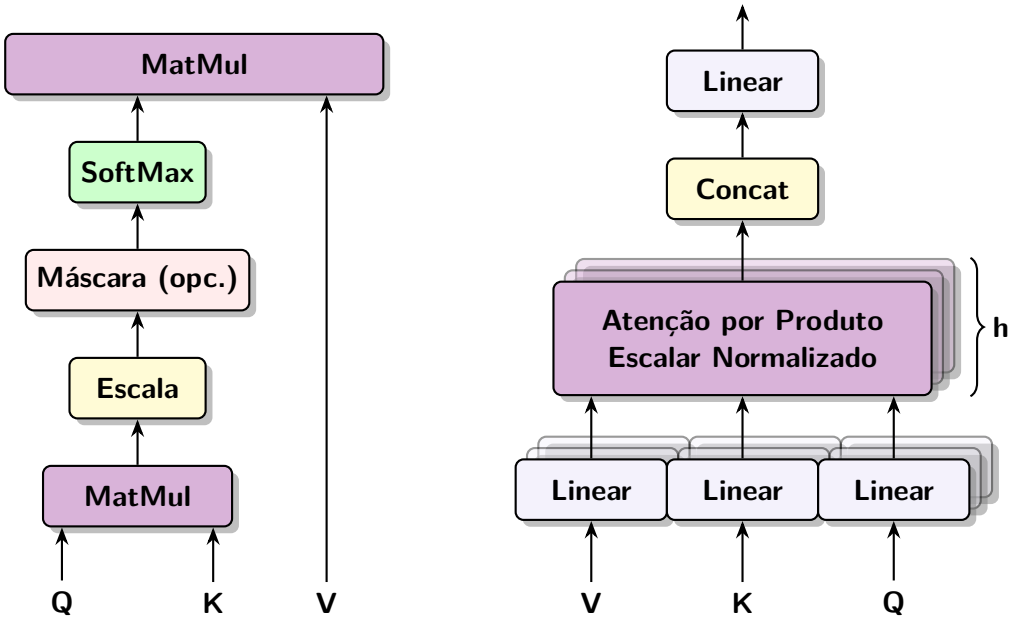


Figura 1 – À esquerda, o funcionamento do mecânismo de atenção por produto escalar normalizado. À direita, o funcionamento do modulo de multi-atenção.

Os vetores descritores do texto passam por várias instâncias do bloco descrito, atravessando também camadas de normalização (LNs) e perceptrons multicamada (MLPs). Ao fim da rede, a maneira em que as representações resultantes de cada *token* são usadas depende do contexto de aplicação. Para geração de texto, como em *chatbots*, o vetor

propagado do último termo gerado até então é empregado no cálculo do próximo item. Já para tradução, o *embedding* da palavra sendo traduzida é usado para gerar sua equivalente em outra língua. Nesse caso, aplica-se uma mascara durante o módulo de atenção para permitir relações apenas com o contexto que precede o *token*, como ilustrado na Figura 1. O modelo também aplica conexões residuais em suas camadas de forma a assegurar ajustes incrementais ao vetor. A arquitetura original do *transformer*, projetada para tradução através de um codificador e decodificador, pode ser vista na Figura 2.

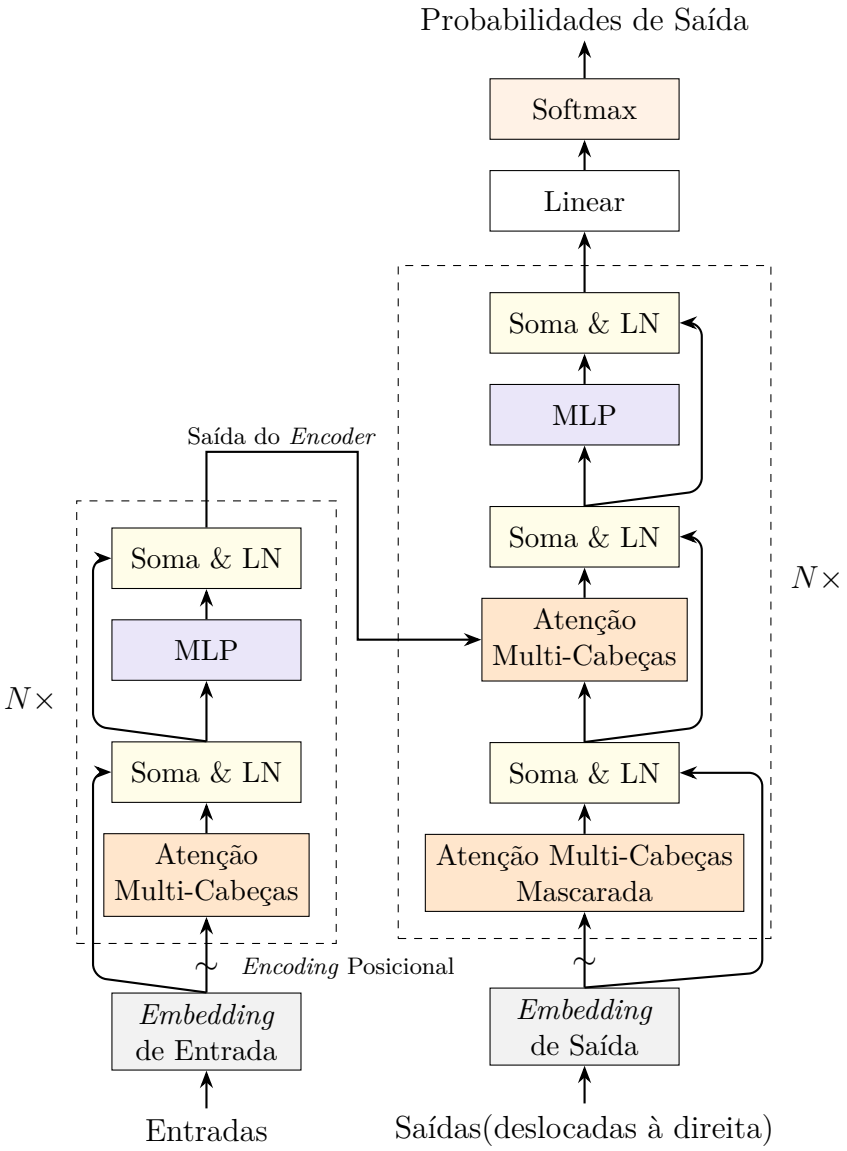


Figura 2 – Arquitetura original do *Transformer*.

2.1.2 *Vision transformers*

A versatilidade dos *transformers* não se limita a texto. Seu funcionamento, centrado no uso de representações vetoriais para captura de traços semânticos ao longo de várias ca-

madas, pode ser facilmente estendido para outros domínios, incluindo visão computacional (VC).

Os *transformers* de visão, ou ViTs, são uma adaptação simples da arquitetura original que permite o processamento de imagens ao invés de palavras. A mudança principal está na definição do *token* processado pelo modelo, que é tratado como um trecho quadrado, ou *patch*, da entrada com arestas de tamanho P. As sub-regiões são achatadas em vetores e mapeadas para a dimensão desejada D por uma camada de projeção linear, sendo então enfileiradas e alimentadas sequencialmente ao modelo.

$$\mathbf{z}_0 = [\mathbf{x}_{\text{classe}}; \mathbf{x}_p^1\mathbf{E}; \cdots; \mathbf{x}_p^{N_p}\mathbf{E}] + \mathbf{E}_{pos}, \qquad \mathbf{E} \in \mathbb{R}^{(P^2 \cdot C) \times D}, \mathbf{E}_{pos} \in \mathbb{R}^{(N+1) \times D} \tag{2.2}$$

$$\mathbf{z}'_\ell = \text{Atenção}(\text{LN}(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1}, \qquad \ell = 1 \ldots L \tag{2.3}$$

$$\mathbf{z}_\ell = \text{MLP}(\text{LN}(\mathbf{z}'_\ell)) + \mathbf{z}'_\ell, \qquad \ell = 1 \ldots L \tag{2.4}$$

$$\mathbf{y} = \text{LN}(\mathbf{z}_L^0) \tag{2.5}$$

O resto do processo segue o mesmo fluxo descrito na seção anterior, representado pelas Equações (2.2)-(2.5), onde $\mathbf{x}_p^i\mathbf{E}$ se trata do *embedding* do i-gésimo trecho da imagem e $\mathbf{z}_\ell$ retrata a saída da camada ℓ. Os vetores de posição $\mathbf{E}_{pos}$ descrevem apenas a posição do pedaço na sequência, tornando a captura de qualquer relação espacial 2D entre *patches* algo a ser aprendido. A única outra mudança feita é a inclusão de um *token* especial de classe $\mathbf{x}_{\text{classe}}$ no inicio do arranjo de entrada $\mathbf{z}_0$, cuja representação aglomerada ao final da rede é usada como saída para tarefas de classificação (2.5).

2.1.3 *Integrando linguagem e visão*

Dada a flexibilidade do *transformer* padrão para aplicações tanto em PLN quanto em VC, torna-se claro seu potencial para multimodalidade: a capacidade para acomodar ambos os domínios simultaneamente. A compreensão de imagens e texto por um único modelo pode ser retratado como um desafio de mapeamento, onde ambos os tipos de dado devem ser co-representados pelo mesmo espaço vetorial, permitindo seu dialogo.

Na maioria dos modelos recentes, isso é atingido pelo uso de um bloco dedicado para a codificação do componente visual da entrada, frequentemente utilizando um ViT. Assim, a abordagem mistura as duas arquiteturas vistas nas seções anteriores, ligando-as por uma camada de adaptação. Essa, geralmente linear, projeta os *tokens* visuais para o domínio textual, permitindo que sejam usados diretamente pelos módulos de atenção, dentre outros, como visto na Figura 3.

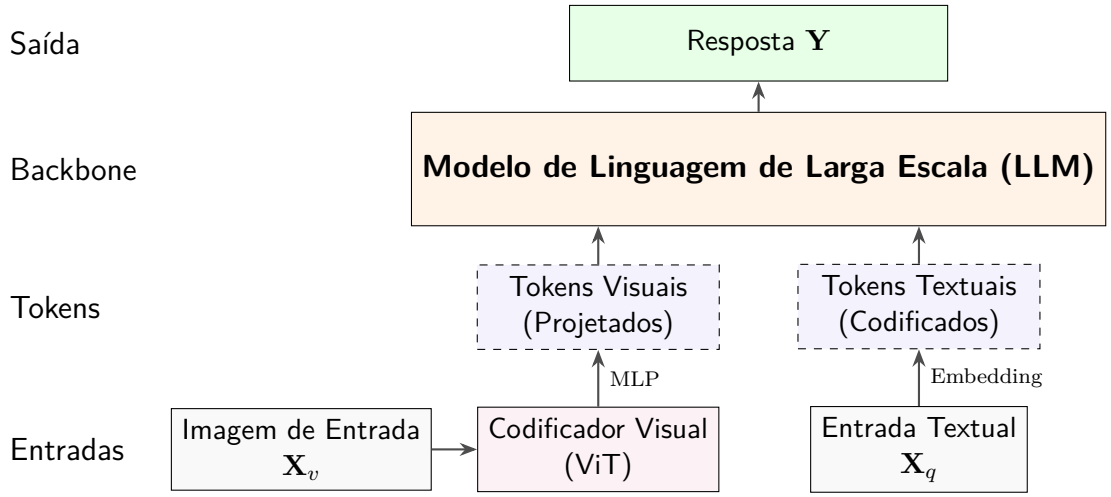


Figura 3 – Fluxo de processamento: modelo de linguagem e visão (VLM).

Naturalmente, para que sejam uteis nas camadas subsequentes, é importante que os *embeddings* gerados capturem uma gama de informações gerais e variadas sobre o conteúdo de cada imagem, tornando desaconselhável o uso de codificadores especializados para um proposito único. A fim de obter-se um modelo capaz de representar cenas de forma abrangente, Radford et al. (2021) optou por um treinamento não supervisionado e de larga escala. Similarmente a Radford et al. (2018) e Radford et al. (2019), o método utilizou um conjunto de 400 milhões de pares de imagens e legendas, onde o modelo deve identificar a descrição correta dentre múltiplas escolhas. Esse formato auto-supervisionado elimina a dependência de dados rotulados manualmente e explora a capacidade do *transformer* em extrair representações semânticas complexas de grandes volumes de dados. Devido à sua eficácia, ViTs treinados com essa abordagem são amplamente adotados como codificadores de imagem em modelos multimodais modernos.

De posse de *tokens* visuais de alta qualidade, a etapa seguinte é capacitar o modelo de linguagem a interpretá-los, aprendendo a extrair informações latentes para executar tarefas multimodais complexas. Para esse fim, diversas abordagens foram exploradas na literatura. Liu et al. (2023a) usa imagens legendadas, alimentando suas descrições para um modelo de linguagem congelado que as transforma em um conjunto de perguntas e respostas. O LLM é então treinado a responder as perguntas usando a entrada fornecida. Outras abordagens, como simultaneamente treinar o modelo para minimizar várias funções de perda envolvendo compreensão visual (LI et al., 2022), também são adotadas. Usualmente, os *embeddings* do ViT são fornecidos de forma sequencial antes dos vetores relativos ao texto. No entanto, recentemente, arquiteturas alternativas que os alimentam de forma distribuída também foram concebidas (MENG et al., 2024). Em algumas implementações (Chameleon Team, 2024), um único modelo misto é adotado para o processamento de dados visuais e textuais.

2.1.4 *Pré-treinamento e ajuste fino*

A popularização dos *transformers* como paradigma arquitetural para modelos de linguagem alterou fundamentalmente o treinamento de tais sistemas. Similar aos codificadores visuais na seção anterior, abordagens puramente supervisionadas vêm perdendo protagonismo devido aos desafios de escalabilidade limitando seu uso para grandes conjuntos de dados e à sua ineficácia na indução de comportamentos generalistas. A dependência de rótulos específicos confina o aprendizado à distribuição de dados vista no treino, impedindo a formação de noções de mundo abrangentes e comprometendo robustez na tarefa-alvo frente a nuances ou variações não mapeadas pela supervisão.

Para mitigar essas limitações, consolidou-se o paradigma de pré-treinamento seguido por ajuste fino, ou *fine-tuning*. Essa abordagem fundamenta-se no uso de tarefas autosupervisionadas em grandes volumes de dados não rotulados para o aprendizado de representações universais, que são então adaptadas para um objetivo-alvo através de supervisão.

Radford et al. (2018) demonstrou que o pré-treino generativo captura tendências linguísticas complexas ao definir um bom ponto de partida para o ajuste, permitindo a transferência do conhecimento geral obtido para diversas tarefas subsequentes. O método consistentemente alcança um ganho coletivo de generalização e performance ao ampliar a compreensão que seria obtida apenas pelo refino. Radford et al. (2019) expandiu sobre esse conceito provando que, para um corpo de dados suficientemente grande, é ainda possível treinar um sistema capaz de atingir o estado da arte em várias aplicações sem a necessidade de qualquer supervisão explícita, demonstrando a eficácia do pré-treinamento na aprendizagem de *embeddings* expressivos e na obtenção de vastas bases de conhecimento geral e específico.

Diante disso, o papel do *fine-tuning* em sistemas modernos transcende a simples especialização em tarefas fechadas, consolidando-se como uma etapa de alinhamento comportamental. Seu foco não é mais a injeção de novas informações, mas sim a otimização da maneira como o modelo navega seu espaço de busca interno. Ele ancora a IA às preferências humanas e facilita a indução de comportamentos como analise lógica e autorreflexão. Métodos recentes efetivam essa função pelo emprego de técnicas de reforço, usando avaliações humana sobre respostas geradas para atingir concordância com os valores e intenções do usuário (OUYANG et al., 2022) ou tarefas com soluções certas e definidas, como a resolução ou dedução de expressões matemáticas, para incentivar a formação de correntes lúcidas de raciocínio. Essas metologias também são ocasionalmente aplicadas na delimitação do formato desejado para as saídas do modelo, sendo um dos meios possíveis para alcançar conformidade ao padrão de conversação visto em *chatbots* contemporâneos.

2.2 *Aprendizado por reforço*

A ampla adoção do aprendizado por reforço (RL) na etapa de ajuste fino responde à necessidade de otimizar objetivos que não são facilmente capturados por funções de perda tradicionais. Para tanto, a abordagem busca ativamente por respostas de maior qualidade dentro de um espaço de possíveis soluções, em vez de apenas copiar exemplos fixos ou imitar padrões de dados. Essa flexibilidade permite que o sistema refine seu comportamento em cenários onde existem vários caminhos corretos, o que viabiliza tanto o alinhamento ético quanto o aprimoramento das capacidades analíticas discutidas anteriormente.

Diferente de outros métodos, o RL define-se como um paradigma baseado em tentativa em erro: um *agente* busca aprender uma *política* de decisões ideal através de interações com um *ambiente* dinâmico. Ao contrário do aprendizado supervisionado, não existem respostas explícitas que o modelo deve prever, apenas *recompensas* retornadas pelo mundo para sinalizar aproximação ao objetivo final. A adesão à essa dinâmica resulta em um algoritmo robusto para problemas nos quais a solução não é estática ou unívoca.

2.2.1 *Processo de decisão de Markov*

O Processo de Decisão de Markov, ou MDP, é a fundação teórica sobre a qual tais métodos são construídos. Ele fornece uma formalização matemática para processos descritos como sequências de decisões, onde o agente decisor escolhe uma dentre múltiplas *ações* possíveis seguindo uma política internamente determinada.

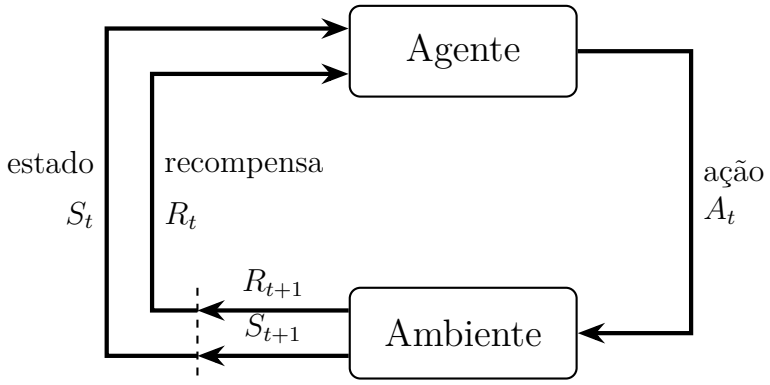


Figura 4 – Interação agente-ambiente em um MDP.

A interação com o ambiente ocorre conforme a Figura 4. Em cada etapa t do processo, o agente recebe uma representação da situação atual, denominada *estado*, e uma *recompensa* com base nele e em sua última ação. Na primeira iteração do *episódio*, recebe-se apenas um estado incial relativo ao ponto de partida.

Na definição formal do MDP, os conjuntos de estados, ações e recompensas possíveis são representados por S, A e R, respectivamente. Caso sejam finitos, o problema trata-se de um MDP finito, onde S e A são variáveis aleatórias com distribuições de probabilidades bem

definidas e dependentes apenas do estado e ação anteriores (Equação 2.6). Tal dependência exclusiva no presente para a previsão do futuro é referida como a *propriedade de Markov* e implica que toda informação relevante para o futuro deve estar presente no estado atual.

$$\sum_{s' \in S} \sum_{r \in R} p(s', r|s, a) = 1 \tag{2.6}$$

O foco do aprendizado por reforço está em aprender uma política π capaz de escolher a ação que maximize o retorno cumulativo a partir do estado atual. A política é definida como uma distribuição sobre todas as ações possíveis (Equação 2.7). Um retorno positivo por parte do ambiente é tratado como análogo ao alcance do comportamento desejado. Assim, é imperativo que esse esteja alinhado com a conduta que se deseja ensinar.

$$\sum_{a \in A} \pi(a|s) = 1 \tag{2.7}$$

Formalmente, diz-se que objetivo final do algoritmo é maximizar o *retorno esperado*, definido na Equação 2.8. Na expressão, T é a última etapa presente no episódio. O conceito de episódio existe para problemas que podem ser divididos em sessões, como partidas em um jogo, locomoção de um robô, e outras tarefas similarmente repetitivas. Nesses casos, o processo chega ao fim ao entrar em um *estado terminal*, definido arbitrariamente.

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \cdots + R_T \tag{2.8}$$

Em casos onde não se há um termino claro ($T = \infty$), enfatiza-se recompensas próximas através de um *fator de desconto* γ, que é multiplicado de forma cumulativa nos retornos seguintes de forma a eventualmente torná-los desprezíveis. Assim, obtêm-se o retorno descontado, expressado na Equação 2.9.

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma \check{s} R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \tag{2.9}$$

De acordo com o objetivo descrito, o valor associado a um dado estado, ou à qualquer agrupamento deste com uma ação nele possível, é definido como o retorno esperado ao fim do episódio seguindo a política atual e desconsiderando recompensas passadas. Tais funções de valor, descritas nas Equações (2.10) e (2.11), denominam-se *função de valor de estado* e *função de valor de ação*, respectivamente. O subscrito π em ambas indica sua dependência na regra de decisão atual.

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s], \qquad s \in \mathcal{S} \tag{2.10}$$

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a], \qquad s \in \mathcal{S}, a \in \mathcal{A} \tag{2.11}$$

2.2.2 *Otimização de política por gradiente*

No treinamento de VLMs e LLMs por reforço, é comum otimizar a política de decisão pelo uso de métodos de gradiente, assim como em outros paradigmas de aprendizagem. Para tanto, é necessária a definição de um objetivo $J(\theta)$ que se deseja maximizar ou minimizar. Aqui, esse é definido como a função de valor $v_\pi(s)$ para o primeiro estado do episodio (s_0), que é análogo à recompensa total recebida (Equação 2.12). Na formula, o simbolo θ refere-se aos parâmetros da política π que se deseja otimizar.

$$J(\theta) \doteq v_\pi(s_0), \quad \theta \in \mathbb{R}^d, s_0 \in \mathcal{S} \tag{2.12}$$

De posse de uma métrica, a atualização dos parâmetros da política é feita com base no gradiente ∇ do objetivo. Diferentemente de outras abordagens, o acréscimo é feito no mesmo sentido do vetor calculado a fim de maximizar a função-alvo (Equação 2.13). O uso de $v_\pi(s)$ como medida de performance, apesar de condizente com a teoria, complica o cálculo direto do gradiente. A distribuição dos estados visitados é sensível a mudanças na política, criando uma dependência implícita dos parâmetros que torna o gradiente da função de valor intratável sem um modelo explícito do ambiente.

$$\theta_{t+1} = \theta_t + \alpha \nabla J(\theta_t), \quad \alpha \in \mathbb{R} \tag{2.13}$$

Para resolver esse impasse, o *teorema do gradiente da política* é aplicado. Ele torna a atualização dos parâmetros independente da dinâmica do ambiente, delimitando o gradiente do objetivo em termos apenas do gradiente da própria política (Equação 2.14). Embora seja uma relação proporcional, sua proporcionalidade é absorvida pelo fator de aprendizado α, viabilizando o mecanismo de atualização dos parâmetros. Na formulação, $\mu(s)$ é uma métrica de ocupação, indicando a duração de permanência em cada estado s durante o episódio. A função de valor de ação $q_\pi(s, a)$ atua como um peso de importância, sinalizando a qualidade da ação tomada, enquanto $\nabla\pi(a|s, \theta)$ define a direção de ajuste nos parâmetros para aumentar ou diminuir a probabilidade dessa ação. Visto que o primeiro somatório pondera a expressão pela distribuição de ocupação dos estados, ele pode ser reescrito como a esperança $\mathbb{E}_\pi$, assim viabilizando a estimativa do gradiente por amostragem.

$$\nabla J(\theta) \propto \sum_s \mu(s) \sum_a q_\pi(s, a) \nabla\pi(a|s, \theta) = \mathbb{E}_\pi \left[\sum_a q_\pi(s, a) \nabla\pi(a|s, \theta) \right] \tag{2.14}$$

A fim de torná-la aplicável na prática, o algoritmo REINFORCE (WILLIAMS, 1992, p. 229) modifica essa base para estimar o gradiente via métodos de Monte Carlo. Em vez de computar a função de valor de ação exata, o agente interage com o ambiente até o fim de um episódio e utiliza o retorno total acumulado, G_t, como uma estimativa estocástica para $q_\pi(s, A_t)$, onde A_t representa a sequência de ações amostrada da trajetória completa conforme π. A atualização dos parâmetros ocorre, portanto, na direção $\nabla J(\theta) \approx$

$\nabla \ \ln \pi(A_t|S_t, \theta)$, o que intuitivamente aumenta a probabilidade de ações que resultaram em altos retornos e penaliza aquelas associadas a recompensas baixas. Por depender de trajetórias completas, o REINFORCE é um método *on-policy* que, embora simples e livre de viés, costuma apresentar alta variância em suas estimativas de gradiente. Para viabilizar a implementação desse cálculo em redes neurais, aplica-se a identidade $\nabla\pi = \pi\nabla \ \ln \pi$, que permite reescrever a esperança em uma forma computacionalmente eficiente.

$$\begin{aligned}\nabla J(\theta) &= \mathbb{E}_\pi \left[\sum_a \pi(a|S_t, \theta) q_\pi(S_t, a) \frac{\nabla\pi(a|S_t, \theta)}{\pi(a|S_t, \theta)}\right] = \mathbb{E}_\pi \left[q_\pi(S_t, A_t) \frac{\nabla\pi(A_t|S_t, \theta)}{\pi(A_t|S_t, \theta)}\right] \\ &= \mathbb{E}_\pi \left[G_t \frac{\nabla\pi(A_t|S_t, \theta)}{\pi(A_t|S_t, \theta)}\right] = \mathbb{E}_\pi \left[G_t \nabla \ln \pi(A_t|S_t, \theta)\right]\end{aligned}$$

$$\theta_{t+1} = \theta_t + \alpha G_t \nabla \ln \pi(A_t|S_t, \theta_t), \qquad t <= T, \alpha \in \mathbb{R} \tag{2.15}$$

Para mitigar o problema da alta variância, é comum a introdução de uma função de linha de base, ou *baseline*, $b(s)$. Ao subtrair-la do retorno G_t, a esperança do gradiente permanece inalterada, mas a variância das atualizações é reduzida, uma vez que o sinal de reforço passa a refletir distância do referencial (Equação 2.16). Alguns algoritmos escolhem a própria função de valor de estado $v_\pi(s)$ como *baseline*, caracterizando-os como métodos *ator-crítico*. Nessas abordagens, o modelo é dividido em dois componentes: o *ator*, que representa a política $\pi(a|s, \theta)$ e decide as ações, e o *crítico*, que aprende a aproximar a função de valor $\hat{v}(s, w)$, onde w são os pesos do estimador. Um caso especial é o *ator-crítico de um passo*, que elimina a dependência de trajetórias completas substituindo o retorno total pela diferença entre a recompensa imediata somada ao valor estimado do próximo estado menos a estimativa do estado atual (Equação 2.17). Diferente do REINFORCE, essa estrutura permite atualizações online a cada passo temporal, aproveitando o aprendizado do crítico para reduzir a variância e acelerar a convergência, embora introduza um viés inicial devido ao uso de estimativas em vez de retornos reais.

$$\theta_{t+1} = \theta_t + \alpha(G_t - b(S_t)) \nabla \ln \pi(A_t|S_t, \theta_t) \tag{2.16}$$

$$\theta_{t+1} = \theta_t + \alpha(R_{t+1} + \gamma\hat{v}(S_{t+1}, w) - \hat{v}(S_t, w)) \nabla \ln \pi(A_t|S_t, \theta_t) \tag{2.17}$$

2.2.3 *Otimização de política proximal*

Dentre as técnicas de aprendizado por reforço da última década, a otimização de política proximal (PPO) (SCHULMAN et al., 2017) é possivelmente a mais influente. Desenvolvida para ambientes robóticos e de navegação, ela tornou-se o componente central em fluxos de aprendizado por reforço por parecer humano (RLHF), sendo uma das técnicas preferidas para o alinhamento de LLMs e VLMs.

Sua concepção foi uma resposta à necessidade de algoritmos que fossem, simultaneamente, estáveis e escaláveis para ambientes de alta complexidade. O principal entrave

das abordagens puramente baseadas no gradiente de política, como o ator-crítico visto na seção anterior, reside em sua extrema sensibilidade ao fator de aprendizado. Em espaços de parâmetros complexos, um passo muito pequeno torna o aprendizado exageradamente lento, enquanto um incremento excessivo pode desestabilizar a política, empurrando-a para regiões de baixo desempenho de forma irreversível. Essa instabilidade exige um ajuste fino e exaustivo dos hiperparâmetros e torna o treinamento imprevisível em ambientes de alta dimensionalidade.

A fim de alcançar maior estabilidade, mudanças devem ser feitas de forma a limitar o tamanho da atualização da política a cada passo. Para tanto, o objetivo de maximização $J(\theta)$ visto antes é reformulado (Equação 2.18). Introduz-se o símbolo $\mathcal{A}_t$, denominado *vantagem*, que formaliza o ganho apresentado no contexto de métodos ator-crítico, delimitando-o como o diferencial em relação à expectativa $\hat{v}(s, w)$ do crítico. Em aplicações modernas, tal esperança é aprendida e estimada por uma rede profunda especializada. No PPO, o cálculo de $\mathcal{A}_t$ segue o método da estimativa generalizada da vantagem (GAE). A abordagem aumenta o estimador de um passo visto na seção anterior através de uma soma ponderada, onde λ regula a influência de passos futuros, permitindo que a vantagem generalize desde o erro de diferença temporal básico até retornos de trajetórias completas (Equação 2.19).

$$J(\theta) = \mathbb{E}_t \left[r_t(\theta) \hat{\mathcal{A}}_t \right], \quad r_t(\theta) = \frac{\pi(A_t|S_t, \theta)}{\pi(A_t|S_t, \theta_{coleta})} \tag{2.18}$$

$$\hat{\mathcal{A}}_t^{GAE}(\gamma, \lambda) = \sum_k (\gamma\lambda)^k \left(r_{t+k} + \gamma \hat{v}(s_{t+k+1}) - \hat{v}(s_{t+k}) \right) \tag{2.19}$$

Além disso, o termo $\ln \pi(A_t|S_t, \theta)$ é substituído por $r_t(\theta)$: uma generalização do termo anterior para o caso *off-policy*, onde a política aprendida não é a mesma usada na coleta de dados. O simbolo representa a razão entre a política π_θ sendo otimizada e a política de referência $\pi_{\theta_{coleta}}$ responsável pela amostragem da trajetória. O PPO faz uso dessa formulação para o uso de técnicas de amostragem por importância. Nelas, $r_t(\theta)$ indica o quanto a nova política desviou da anterior após uma atualização de parâmetros. Caso seu valor seja 1, implica-se que não houve mudança na distribuição de ações prevista. Valores maiores indicam que uma dada ação tornou-se mais provável sob a nova política, enquanto menores apontam o oposto. Tal métrica serve como base para monitorar a magnitude das mudanças na estratégia do agente, permitindo a interferência em atualizações excessivamente bruscas e instáveis.

O uso da razão acima para o controle do aumento do ganho é o que possibilita ao PPO atingir estabilidade de forma computacionalmente eficiente, superando abordagens com restrições complexas (SCHULMAN et al., 2015). O algoritmo modifica o objetivo, cortando-o com base no valor de $r_t(\theta)$ e no sinal da vantagem (Equação 2.20). A função-alvo resultante, $J^{CLIP}(\theta)$, é limitada pelo hiperparâmetro ϵ de forma a restringir a razão $r_t(\theta)$ a ser menor que $1 + \epsilon$ para ganhos positivos e torná-la livre para ganhos negativos. Dessa

forma, a técnica impede incrementos exagerados enquanto pune ações ruins de forma irrestrita.

$$J_{CLIP}(\theta) = \mathbb{E}_t \left[\min(r_t(\theta)\hat{\mathcal{A}}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{\mathcal{A}}_t) \right] \tag{2.20}$$

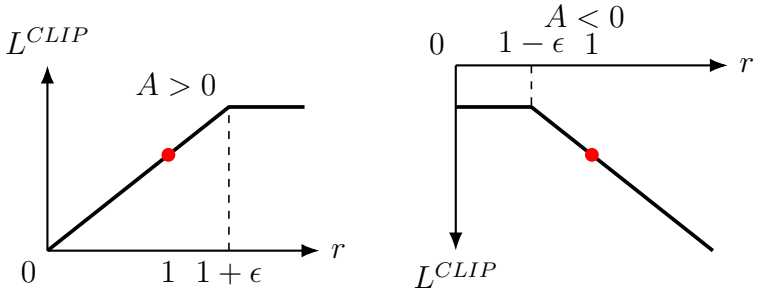


Figura 5 – Função objetivo substituta clipada do PPO para vantagens positivas e negativas.

2.2.4 *Otimização de política relativa ao grupo*

Apesar das melhorias por ele introduzidas, o PPO ainda possui uma grande desvantagem: a função de valor de estado $\hat{v}(s, w)$ por ele estimada para servir como crítico requer o treinamento e uso de um modelo com tamanho comparável à política de decisão. Isso, por sua vez, traz consigo um maior consumo de memória e computação.

Para contornar esse problema, Shao et al. (2024) propõe a otimização de política relativa ao grupo (GRPO), que descarta o uso de um estimador como crítico. Dado um mesmo *prompt* e um único passo, o método amostra múltiplas saídas ($\mathbf{r}$), usando a média do grupo como *baseline* para cálculo de $\mathcal{A}_t$ e o desvio padrão para normalização a escala (Equação 2.21). A fim de tratar trajetórias completas, a abordagem gera múltiplos caminhos em paralelo, usando as recompensas totais das sequências ($\mathbf{R}$) para padronizar os retornos individuais. A vantagem a cada etapa é então calculada como o somatório dos valores subsequentes, conforme visto na Equação 2.22. Nela, g refere-se à trajetória amostrada e k ao passo.

$$\hat{\mathcal{A}}_t = \tilde{r_g} = \frac{r_g - \textit{média}(\mathbf{r})}{\textit{desvio}(\mathbf{r})}, \quad \mathbf{r} = \{r_1, r_2, \ldots, r_G\} \tag{2.21}$$

$$\hat{\mathcal{A}}_t = \sum_{k=t}^{K_g} \tilde{r_g}^k, \quad \tilde{r_g}^k = \frac{r_g^k - \textit{média}(\mathbf{R})}{\textit{desvio}(\mathbf{R})}, \quad \mathbf{R} = \{\{r_1^1, \ldots, r_1^{K_1}\}, \ldots, \{r_G^1 \ldots, r_G^{K_G}\}\} \tag{2.22}$$

O objetivo de maximização é então reformulado como $J_{GRPO}(\theta)$, complementando o J_{CLIP} usado no PPO pelo uso da divergência de Kullback-Leibler (KL) (KULLBACK; LEIBLER, 1951), que calcula a distância entre distribuições de probabilidade. Aqui, ela é deduzida do objetivo de forma a punir atualizações muito grandes, sendo escala pelo hiperparâmetro β. A função-alvo também faz uso da nova formulação de vantagem, definindo-se em função do ganho médio entre as G amostras coletadas a cada episódio. A

expectativa na Equação 2.23 encapsula o processo de amostragem em grupo: para cada instrução ou comando q, o modelo gera simultaneamente G trajetórias distintas.

$$J_{GRPO}(\theta) = \mathbb{E}_{q\sim P(q),\{A_i\}_{i=1}^{G}\sim\pi_{\theta_{coleta}}} \left[\frac{1}{G}\sum_{i=1}^{G} J_{CLIP}(\theta) - \beta D_{KL}(\pi_\theta||\pi_{coleta}) \right] \tag{2.23}$$

2.2.5 *Otimização de política de sequência relativa ao grupo*

Embora o GRPO reduza o custo computacional ao dispensar o crítico, Zheng et al. (2025) apontam que sua formulação herda uma fragilidade do mecanismo de amostragem por importância. A razão $r_t(\theta)$ é aplicada de forma independente a cada token e estimada a partir de uma única amostra por posição. Em contraste, a recompensa e a vantagem são atribuídas à sequência inteira. A diferença de granularidade entre as duas métricas introduz um ruído de alta variância que se acumula com o comprimento da resposta e é amplificado pelo mecanismo de corte, podendo levar ao colapso irreversível do modelo. O efeito é particularmente severo em arquiteturas de mistura de especialistas (MoE), nas quais pequenas atualizações de parâmetros alteram o conjunto de especialistas ativados para uma mesma resposta, desestabilizando ainda mais as razões calculadas por token.

A otimização de política de sequência relativa ao grupo (GSPO) corrige esse problema ao alinhar a unidade da razão de importância à unidade da recompensa. Em vez de operar por token, define-se uma única razão $s_i(\theta)$ para a sequência completa, baseada em sua probabilidade conjunta e normalizada pelo seu comprimento $|A_i|$ (Equação 2.24). A normalização mantém $s_i(\theta)$ em uma faixa numérica estável independentemente do tamanho da resposta e reduz a variância das atualizações.

$$s_i(\theta) = \left(\frac{\pi_\theta(A_i \mid q)}{\pi_{\theta_{coleta}}(A_i \mid q)} \right)^{\frac{1}{|A_i|}} = \exp\left(\frac{1}{|A_i|}\sum_{t=1}^{|A_i|} \ln \frac{\pi_\theta(A_{i,t} \mid q, A_{i,<t})}{\pi_{\theta_{coleta}}(A_{i,t} \mid q, A_{i,<t})} \right) \tag{2.24}$$

O objetivo de maximização preserva a estrutura cortada do PPO, porém com o corte aplicado à sequência inteira e não a tokens individuais (Equação 2.25). A vantagem $\hat{\mathcal{A}}_i$ corresponde à mesma normalização relativa ao grupo apresentada na Equação 2.21, agora atribuída de modo uniforme a toda a resposta. Por depender apenas da probabilidade conjunta agregada da sequência, o GSPO mostra-se robusto à volatilidade de ativação dos especialistas em modelos MoE, além de ser mais tolerante a discrepâncias numéricas entre os mecanismos de inferência e treinamento. Essas propriedades fizeram do método a base do treinamento por reforço dos modelos Qwen3 (BAI et al., 2025b).

$$J_{GSPO}(\theta) = \mathbb{E}_{q\sim P(q),\,\{A_i\}_{i=1}^{G}\sim\pi_{\theta_{coleta}}} \left[\frac{1}{G}\sum_{i=1}^{G} \min\left(s_i(\theta)\hat{\mathcal{A}}_i,\ \text{clip}(s_i(\theta), 1-\epsilon, 1+\epsilon)\hat{\mathcal{A}}_i \right) \right] \tag{2.25}$$

2.3 *Modelos para uso de interface gráfica*

A operação autônoma de interfaces gráficas de usuário (GUIs) ocupa uma posição intermediária entre os *chatbots* puramente textuais e os agentes multimodais encorpados. Onde tentativas anteriores esbarravam na rigidez de heurísticas manuais e seletores frágeis, os LLMs e VLMs trouxeram uma capacidade de percepção e raciocínio que viabiliza o sucesso em tarefas de maior complexidade. Navegar esses ambientes exige mais do que a identificação passiva de elementos. O agente precisa traduzir objetivos abstratos em sequências lógicas de ações dentro de um espaço dinâmico e frequentemente imprevisível, atuando como um mediador que converte pixels e texto em ações produtivas.

2.3.1 *Navegação baseada em frameworks*

A primeira onda de abordagens para o uso de interfaces gráficas apoiou-se em modelos de linguagem de propósito geral, como o GPT-4 ou o Claude, inseridos em estruturas responsáveis pela mediação entre o modelo e o ambiente operacional (GUR et al., 2023; ZHOU et al., 2024). Esses *frameworks* operam por um ciclo de orquestração no qual a tela é primeiro convertida em uma representação textual ou estruturada, recorrendo a árvores de acessibilidade, *Document Object Models* (DOM) ou HTML puro no caso de aplicações web (DENG et al., 2023). A tradução permite que um modelo treinado apenas com texto compreenda a hierarquia e as propriedades dos elementos sem depender de percepção visual direta. Sobre essa representação, a navegação ocorre por engenharia de *prompt*, combinando a descrição da interface, um conjunto de ferramentas externas e poucas demonstrações do comportamento desejado (*few-shot prompting*), com a decisão guiada por cadeias de raciocínio (*chain-of-thought*) antes da emissão do comando executado por *software* como Playwright ou Selenium.

A modularidade viabiliza o uso de LLMs de última geração, mas ao custo de uma dependência crítica da qualidade dos metadados. Árvores de acessibilidade mal implementadas ou ausentes anulam a capacidade de navegação, e o pré-processamento acarreta custos adicionais de computação e latência. A externalização das diretivas de navegação, codificadas manualmente por desenvolvedores, agrava a fragilidade. Alterações de *layout* ou de cenário exigem a reescrita das regras, o que eleva o custo de manutenção e limita a escalabilidade em ambientes dinâmicos e imprevisíveis. A representação puramente textual da tela ainda descarta pistas visuais que sinalizam funcionalidade sem correspondente textual, como ícones.

2.3.2 *Modelos nativos de navegação*

A superação das barreiras dos *frameworks* de orquestração apoiou-se em dois marcos. O primeiro foi a adoção de VLMs como ponto de partida, o que deslocou a operação baseada

em metadados para um paradigma de percepção visual bruta e dispensou representações intermediárias como o DOM (CHENG et al., 2024; XU et al., 2024). Abordagens híbridas que combinavam ambos foram propostas, mas rapidamente suplantadas (ZHENG et al., 2024). Nesse arranjo, o modelo interpreta a interface em sua totalidade e trata a navegação como uma tarefa mista de uso de ferramentas e de posicionamento visual (*grounding*), emitindo comandos textuais associados diretamente a coordenadas de *pixel*.

O segundo marco foi o treinamento explícito desses modelos em cenários de navegação. Em vez de partir de um VLM genérico, expõe-se o modelo a conjuntos de dados que reúnem capturas de tela, trajetórias de interação humana e anotações de elementos, o que consolida internamente a correspondência entre instrução, comando de saída e localização do alvo em um espaço de ação unificado. Tratar percepção e decisão como um processo único, ajustável por parâmetros compartilhados, confere habilidades generalizáveis de navegação e robustez frente a variações de *layout* por analogia estrutural. As tentativas iniciais instruíram esses padrões por supervisão direta, obtendo rótulos de *grounding* a partir do código-fonte de páginas web e coletando trajetórias anotadas por humanos para o aprendizado da navegação (HONG et al., 2023; LI et al., 2025).

2.3.3 *Do ajuste supervisionado ao reforço*

O ajuste fino supervisionado (SFT) para o treinamento de agentes de navegação, apesar de suas melhorias, esbarra em um gargalo de escalabilidade e generalização por sua dependência de conjuntos de dados exaustivamente rotulados por humanos. Essa limitação impõe um custo elevado à criação de bases compreensivas e introduz um viés de exposição, restringindo o aprendizado do modelo e potencialmente levando a falhas catastróficas diante de cenários nunca antes vistos.

Acompanhando o sucesso do DeepSeek-R1 (DeepSeek-AI, 2025) no campo dos LLMs, a adoção de métodos baseados em aprendizado por reforço mitiga esse problema ao substituir a imitação de rótulos por sinais de recompensa fundados na corretude do passo ou na completude da tarefa. A técnica simplifica a geração de dados, pois requer apenas a trajetória necessária para concluir o objetivo, e dispensa a supervisão densa de cada ação. Ao incentivar o raciocínio pelo *prompt* em vez do aprendizado por imitação, o modelo torna-se livre para gerar monólogos internos que decidem a ação seguinte, o que favorece a emergência de cadeias de pensamento e permite ao agente aprender sua própria lógica de navegação de forma a aumentar a probabilidade de sucesso na tarefa final.

2.4 *Trabalhos relacionados*

O ajuste de agentes de GUI por aprendizado por reforço com recompensas verificáveis consolidou-se como uma alternativa ao SFT, e os trabalhos que seguem essa linha definem

o contexto imediato desta pesquisa. Os métodos mais próximos compartilham a base multimodal, a formulação da recompensa por regras e o foco na predição de ações, diferindo no domínio-alvo, no algoritmo de otimização ou no regime de treino adotado.

2.4.1 *Recompensas verificáveis no ajuste de agentes de GUI*

Lu et al. (2025b) estão dentre os primeiros a transpor o aprendizado por reforço por regras, no estilo do DeepSeek-R1, para a predição de ações em GUIs. O método proposto, o UI-R1, define uma recompensa unificada que avalia simultaneamente o modelo em relação à corretude do tipo de ação desejada, à precisão dos cliques previstos e à aderência ao formato de saída esperado. A formulação é otimizada via GRPO em um conjunto reduzido de tarefas móveis e sobre uma base compacta. O trabalho evidencia a eficiência em dados do paradigma, alcançando ganhos expressivos de predição com uma fração das amostras exigidas por abordagens supervisionadas.

O GUI-R1 (LUO et al., 2025) estende a proposta para tarefas de alto nível em múltiplas plataformas, incluindo *Windows*, *Linux* e *MacOS*. A abordagem modela um espaço de ação unificado e valida as respostas por uma recompensa que combina a conformidade de formato com a corretude da ação, também sob GRPO e com poucos milhares de exemplos. A formulação de passo único com histórico por etapa e a estrutura de recompensa por regras nela introduzida tornaram-se referência para trabalhos subsequentes de ajuste de agentes por reforço.

Variações posteriores exploram diferentes pontos desse espaço de projeto. O InfiGUI-R1 (LIU et al., 2025) antecede a otimização por reforço com uma fase de *cold-start* por SFT, promovendo a transição de um agente reativo para um deliberativo. O AgentCPM-GUI (ZHANG et al., 2025) aplica o ajuste por reforço a um modelo compacto voltado a ambientes móveis. O GUI-G1 (ZHOU et al., 2025), por sua vez, concentra-se no *grounding* estilo R1 e identifica fragilidades no desenho da recompensa e no papel da normalização por comprimento das gerações.

Em paralelo a essas abordagens *off-policy*, uma linha distinta treina agentes de *desktop* por reforço *online* em ambientes executáveis, como no ComputerRL (LAI et al., 2025), obtendo exploração dinâmica ao custo de uma infraestrutura de execução em larga escala.

2.4.2 *Projetos abertos e modelos nativos de desktop*

O refino por reforço discutido até aqui parte de uma base multimodal já capaz de perceber a interface e emitir ações válidas. Tanto essa base quanto os dados que sustentam o treino dos agentes costumam originar-se de projetos abertos de larga escala, apoiados em ajuste supervisionado. O OpenCUA (WANG et al., 2025) é um desses projetos, articulando

uma infraestrutura de anotação, uma base de dados e modelos treinados de ponta a ponta. Seu conjunto de dados, o AgentNet, reúne dezenas de milhares de trajetórias humanas em três sistemas operacionais e centenas de aplicações e sites, com raciocínio reflexivo em cadeia longa acoplado a cada passo.

Modelos nativos treinados em larga escala compõem o pano de fundo desses projetos. O UI-TARS (QIN et al., 2025) representa um agente fim a fim treinado sobre grandes volumes de demonstração humana, enquanto o OS-Atlas (WU et al., 2025) concentra-se na construção de uma base ampla de *grounding* de elementos. Ambos consolidam a viabilidade da percepção visual bruta como fundamento para a navegação autônoma.

2.4.3 *Posicionamento da proposta*

A Tabela 1 organiza os trabalhos discutidos segundo os eixos que caracterizam cada abordagem, situando a proposta desta pesquisa entre eles.

Tabela 1 – Comparação dos trabalhos relacionados segundo os eixos de projeto.

Trabalho	**Base**	**Algoritmo**	**Regime**	**Domínio**	**Formulação**	**Recompensa**
UI-R1	Qwen2.5-VL 3B	GRPO	*off-policy*	Móvel	Passo único	Regras
GUI-R1	Qwen2.5-VL 3–7B	GRPO	*off-policy*	Multi	Passo único	Regras
InfiGUI-R1	Qwen2.5-VL 3B	SFT + GRPO	*off-policy*	Multi	Passo único	Regras
AgentCPM-GUI	MiniCPM-V 8B	GRPO	*off-policy*	Móvel	Passo único	Regras
OpenCUA	Qwen2.5-VL 7–72B	SFT	—	*Desktop*	Trajetória	—
ComputerRL	GLM-4 9B	GRPO	*online*	*Desktop*	Trajetória	Sucesso
Este trabalho	**Qwen3-VL 4B**	**GSPO**	***off-policy***	***Desktop***	**Passo único**	**Regras**

O conjunto dos trabalhos relacionados revela um ponto pouco ocupado do espaço de projeto. As abordagens por reforço por regras concentram-se em ambientes móveis (LU et al., 2025b; ZHANG et al., 2025) ou distribuem-se por múltiplas plataformas sem especialização em *desktop* (LUO et al., 2025). As fundações voltadas a *desktop* apoiam-se em ajuste supervisionado (WANG et al., 2025; QIN et al., 2025), e as propostas de reforço em *desktop* operam em regime *online* de larga escala (LAI et al., 2025). Este trabalho combina o domínio de *desktop*, o regime *off-policy* de passo único, a base compacta e o uso do GSPO como algoritmo de otimização, um arranjo que nenhum dos trabalhos levantados reúne simultaneamente. A adoção do GSPO distingue-se do GRPO predominante nessa literatura e alinha o refino ao mecanismo de otimização empregado no treinamento dos modelos Qwen3.

3 METODOLOGIA EXPERIMENTAL

Este capítulo descreve a configuração experimental adotada para o treinamento e a avaliação de um CUA compacto sob a restrição de uso exclusivo de *hardware* comercial. A organização segue as diretrizes de Antony (2023) para o planejamento e a condução de experimentos, adaptadas ao contexto computacional deste trabalho. Definem-se, nesta ordem, o objetivo mensurável do experimento, as variáveis de resposta e seus instrumentos de medição, os componentes mantidos fixos, a preparação dos dados de treino, as configurações executadas e o plano de análise dos resultados.

3.1 Definição do problema e objetivo do experimento

O experimento baseia-se na seguinte questão mensurável: verificar se o ajuste fino por aprendizado por reforço sobre dados pré-amostrados melhora o desempenho de um VLM compacto em tarefas de uso de computador sob restrições de recursos. A comparação central aqui abordada confronta o modelo ajustado com a rede base equivalente, ambos avaliados sob condições idênticas nas respostas finais definidas na Seção 3.2.1.

Na descrição do experimento, adota-se a nomenclatura usual de experimentos planejados, em que fatores denotam as variáveis de entrada configuradas pelo experimentador e respostas denotam as grandezas medidas como resultado.

A verificação envolve dois níveis distintos. Durante o treinamento, otimiza-se uma recompensa calculada sobre as ações registradas nas amostras de treino, a qual serve como *proxy* para a melhora do modelo. O objetivo do experimento, contudo, é definido sobre o desempenho nos *benchmarks* de avaliação, aferido de forma independente da recompensa.

O escopo operacional delimita-se a ambientes de *desktop*, a uma única família de modelos e ao regime *off-policy*, conforme os fatores fixados na Seção 3.4.

3.2 Variáveis de resposta

As variáveis de resposta deste experimento organizam-se em dois níveis. As respostas finais medem diretamente o objetivo definido na Seção 3.1, correspondendo ao desempenho do modelo nos *benchmarks* de avaliação. As respostas de processo, por sua vez, são observadas ao longo do treinamento, com papel diagnóstico e de monitoramento. A distinção decorre da natureza do regime adotado. A transferência das melhorias nas respostas de processo para as respostas finais é um efeito que o experimento se propõe a verificar.

3.2.1 Respostas finais

Dois ambientes de avaliação compõem as respostas finais, medindo capacidades complementares do agente. O ScreenSpot-Pro (LI et al., 2025) afere a capacidade de ancoramento espacial. Ele reúne capturas de tela em alta resolução de aplicações de *desktop* voltadas para o mercado profissional. Cada imagem acompanha uma instrução que identifica um elemento-alvo. A resposta reportada é a taxa de acerto: a fração de instruções para as quais a coordenada prevista pelo modelo intersecta a região do elemento-alvo. Por operar sobre capturas fixas, a avaliação é determinística dadas as configurações de inferência.

O OSWorld-Verified (XIE et al., 2024) mede o sucesso na execução de tarefas completas. O *benchmark* instancia ambientes reais de *desktop* em máquinas virtuais, nos quais o agente executa ações em sequência até sinalizar o encerramento. Rotinas de verificação inspecionam o estado final do sistema para determinar o cumprimento do objetivo. A resposta reportada é a taxa de sucesso sobre o conjunto de tarefas. Diferentemente do ScreenSpot-Pro, a medição ocorre em ambiente dinâmico, sujeito a variações de tempo de resposta e de estado entre execuções. Cada avaliação de *benchmark* deste trabalho foi conduzida em passagem única sobre o conjunto de tarefas, sem repetições da medição. As implicações dessa execução única são discutidas na Seção 4.6.

3.2.2 Respostas de processo

Três grandezas principais são acompanhadas ao longo do treinamento. A primeira é a recompensa média por passo, observada tanto em seu valor total quanto decomposta em seus componentes, o que permite discriminar avanços na obediência ao formato de saída de avanços na exatidão das ações (Seção 3.3). A segunda é o comprimento das gerações, que difere do limite máximo de sequência: enquanto o limite é um fator configurável do experimento, o comprimento efetivamente produzido é uma consequência do comportamento do modelo. A terceira é a dispersão das recompensas entre as gerações produzidas para uma mesma amostra, grandeza relevante ao GSPO, cujo sinal de aprendizado depende da existência de variância dentro de cada grupo de gerações.

Essas grandezas não são controladas diretamente e emergem da interação entre o modelo, os dados e a configuração de treino. Seu acompanhamento permite monitorar o andamento da otimização, caracterizar seu comportamento e revelar instabilidades.

3.3 Formulação da recompensa

A otimização da política é controlada por uma função de recompensa multicritério, a qual é estruturada para avaliar simultaneamente a obediência ao formato de saída

desejado e a exatidão das ações preditas. Essa função constitui o instrumento principal de medição do treinamento, definindo o que conta como acerto em relação ao rótulo de cada amostra. Baseada na formulação do GUI-R1 (LUO et al., 2025), a recompensa total R_{passo} distribuída a cada passo é definida pela equação:

$$R_{passo} = w_1 \cdot R_{formato} + w_2 \cdot (R_{tipo} + R_{acerto}) \tag{3.1}$$

em que $R_{formato}$ denota a recompensa de formato, R_{tipo} a recompensa referente ao tipo da ação, e R_{acerto} a recompensa pela ação completa, o que inclui a corretude dos argumentos fornecidos. Todas as subrecompensas operam no intervalo $[0, 1]$. Os coeficientes de ponderação são fixados empiricamente em $w_1 = 0{,}6$ e $w_2 = 0{,}2$ para todas as execuções.

3.3.1 Recompensa de formato ($R_{formato}$)

Atribui-se pontuação máxima ($R_{formato} = 1$) exclusivamente às saídas que respeitam um formato estrito de marcadores. A resposta gerada deve conter, obrigatoriamente e na ordem correta, os blocos `<observation>...</observation>`, `<reason>...</reason>` e `<answer>...</answer>`. O conteúdo do bloco `<answer>` deve constituir uma lista de dicionários, em que cada elemento apresenta uma descrição da ação em linguagem natural e a respectiva chamada de função. A Figura 6 ilustra esse formato. Qualquer desvio dessa sintaxe, violação da assinatura esperada ou previsão fora do espaço de ações acarreta a perda integral dessa subrecompensa.

A verificação dessa subrecompensa apoia-se em regras simples de *regex*, que auditam a presença e a integridade das marcações *XML* na saída gerada.

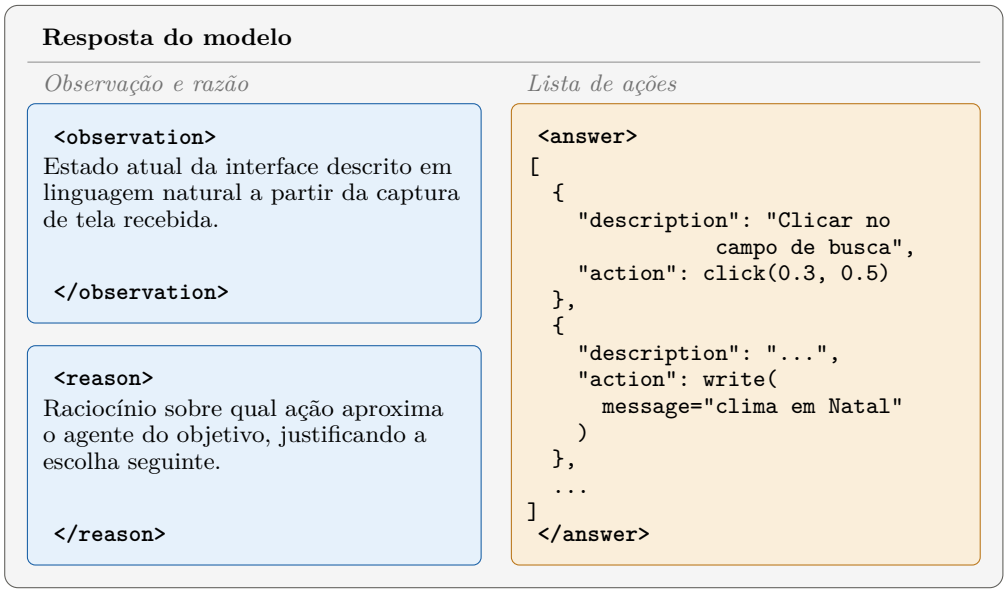


Figura 6 – Formato estrito exigido pela recompensa de formato. A coluna da esquerda contém os blocos `<observation>` e `<reason>` em linguagem natural, enquanto a coluna da direita apresenta o bloco `<answer>`, formatado como uma lista de dicionários em que cada elemento reúne uma descrição da ação e a respectiva chamada de função.

3.3.2 Recompensa de tarefa (R_{tipo} e R_{acerto}) e avaliação sequencial

A fim de avaliar as interações feitas pelo modelo, deve-se considerar sua capacidade de prever múltiplas ações em uma única inferência. Por isso, as predições são validadas usando como *ground truth* a lista de todas as ações futuras restantes no episódio pré-amostrado. Para N ações previstas pelo modelo, estas são pareadas com as N primeiras ações do *ground truth*.

Para a atribuição das recompensas R_{tipo} e R_{acerto}, aplica-se uma regra de acerto sequencial estrito. Ações realizadas sobre uma interface são interdependentes: um erro em um passo intermediário pode invalidar a aplicabilidade das etapas seguintes. Portanto, as ações propostas pelo modelo são validadas em ordem, devendo acertar tanto o tipo da ação ($R_{tipo} = 1$) quanto seus argumentos ($R_{acerto} = 1$). Na ocorrência da primeira divergência em relação ao *ground truth*, a avaliação é interrompida. Todas as predições subsequentes à falha recebem pontuação zero, independentemente de estarem individualmente corretas em relação à lista original, enquanto as anteriores recebem recompensa 1. O ponto de falha é avaliado isoladamente, podendo receber uma recompensa parcial maior que zero, como nos casos em que o modelo acerta o tipo da ação ($R_{tipo} = 1$), mas erra os argumentos ($R_{acerto} = 0$).

A pontuação final repassada aos componentes R_{tipo} e R_{acerto} é a média sobre o total de N ações previstas. Por exemplo, ao propor três ações (x, y, z), se o modelo acerta x e z, mas comete um erro em y, a ação z é invalidada pela quebra da ordem causal.

A recompensa concedida será baseada apenas no acerto de x, resultando em um valor final diluído de 1/3. Esse mecanismo de penalização desencoraja predições excessivamente longas sob incerteza, forçando o agente a gerar sequências múltiplas apenas quando detém alta probabilidade de acerto em toda a cadeia de ações.

3.3.3 Tolerâncias de validação

Para determinar o acerto estrito de uma ação ($R_{acerto} = 1$) e, simultaneamente, acomodar erros de arredondamento ou pequenas variabilidades na predição, a validação incorpora limiares de tolerância específicos por categoria funcional. Esses limiares operam sobre representações padronizadas das ações, distribuídas sobre as categorias do espaço de ações escolhido (Seções 3.5.2 e 3.4.5).

Para ações de ponteiro, dada a ausência de *bounding boxes* no *dataset* e a precisão absoluta dos pontos originais (Seção 3.5.2), estabelece-se um raio de tolerância de 8 *pixels* ao redor do alvo. Esse mecanismo compensa imprecisões de conversão e exige uma acurácia realista do modelo, evitando a obrigatoriedade de inferências em nível de subpixel. No caso de ações de rolagem, adota-se uma tolerância de 16 *pixels* nos deslocamentos verticais e horizontais. Inserções textuais realizadas com comandos de escrita (`write`) são avaliadas pelo *score* F1 entre o texto gerado e a referência original, exigindo-se um limiar mínimo de 0,5 para a confirmação do acerto.

A definição de tolerâncias adaptadas à natureza de cada comando mitiga o risco de superajuste a particularidades numéricas do conjunto de dados. Ao acomodar variações estocásticas e erros de conversão, a validação prioriza a corretude funcional da ação em relação à referência, em vez de exigir uma reprodução determinística e irrealista.

3.4 Componentes fixos do experimento

Definidas as respostas e seu instrumento de medição, descrevem-se os fatores mantidos constantes em todas as configurações executadas. Trata-se das decisões de projeto que condicionam as respostas medidas e delimitam, ao mesmo tempo, o escopo de generalização dos resultados. Mantê-las constantes permite atribuir as diferenças observadas entre as configurações aos fatores que variam.

3.4.1 Modelo base

Para o desenvolvimento de um CUA compacto sob recursos limitados, a escolha da arquitetura da rede é uma decisão essencial. Ela não só condiciona o teto de desempenho atingível, como também a compatibilidade com a infraestrutura adotada e a conformidade

com abordagens consolidadas na literatura. Neste trabalho, adota-se o Qwen3-VL (BAI et al., 2025b) como base para todas as configurações experimentais.

A motivação primária dessa escolha reside na coesão com a literatura recente de agentes usuários de GUIs. Ao longo de 2024 e 2025, a família de modelos Qwen-VL consolidou-se como a escolha predominante para o desenvolvimento de CUAs de código aberto. Em 2024, trabalhos seminais como o SeeClick (CHENG et al., 2024) adotaram o Qwen-VL, enquanto o Aguvis (XU et al., 2024) e o UI-TARS (QIN et al., 2025) foram construídos sobre o Qwen2-VL (WANG et al., 2024). Com o lançamento do Qwen2.5-VL em 2025 (BAI et al., 2025a), este se popularizou rapidamente como a alternativa preferida para a maior parte dos trabalhos subsequentes. Dentre suas aplicações, incluem-se abordagens fundamentadas em aprendizado por reforço diretamente relacionadas a esta pesquisa, como o UI-R1 (LU et al., 2025b) e o GUI-R1 (LUO et al., 2025), bem como o OpenCUA (WANG et al., 2025), do qual deriva o *dataset* adotado neste trabalho.

A adesão a essa linhagem favorece a reprodutibilidade e a comparabilidade dos resultados, uma vez que grande parte dos *benchmarks*, receitas de treino e ferramentas de avaliação do campo foram construídos e calibrados incluindo essa família de modelos. A disponibilidade de pesos abertos em múltiplos tamanhos também alinha a escolha do modelo à proposta central deste trabalho. Assim, a opção pelo Qwen3-VL torna-se a evolução metodológica natural para esta pesquisa.

Dentre os motivos para a adoção ampla da família Qwen para CUAs, destaca-se o seu treinamento nativo para o seguimento de instruções e o uso de ferramentas, capacidades que se alinham diretamente ao paradigma de agentes e diminuem a distância entre o modelo pré-treinado e o comportamento desejado. Soma-se a isso o emprego de processamento visual em resolução dinâmica nativa: ao contrário de abordagens tradicionais que distorcem as capturas de tela para ajustá-las a dimensões fixas, o modelo processa as imagens em suas proporções originais, preservando detalhes essenciais e garantindo precisão no reconhecimento de texto (WANG et al., 2024). Por último, ressalta-se sua proficiência arquitetural em ancoramento espacial. O alto desempenho da rede em correlacionar instruções textuais com elementos visuais na tela é o resultado de um treinamento explícito voltado à localização espacial precisa, uma capacidade que foi sistematicamente aprofundada e reforçada na versão Qwen3-VL (BAI et al., 2025b).

Apesar de não serem exploradas neste trabalho, outras famílias de VLMs de pesos abertos também serviram de base para pesquisas de CUAs. O CogAgent (HONG et al., 2024), pioneiro entre os modelos especializados em GUIs, conta com cerca de 18 bilhões de parâmetros, porte incompatível com a proposta de um agente compacto. Famílias como InternVL (CHEN et al., 2024b) e LLaVA (LIU et al., 2023b) carecem de resolução dinâmica nativa. Alternativas compactas como o MiniCPM-V (YAO et al., 2024), base do agente AgentCPM-GUI (ZHANG et al., 2025), concentram-se em ambientes móveis, em

desacordo com o escopo de *desktop* desta pesquisa.

O presente trabalho optou por não considerar outras arquiteturas, decisão que reflete não apenas as limitações técnicas de cada alternativa, mas a predominância que o Qwen exerce no campo. Trabalhos construídos sobre outras bases reforçam essa posição: o OS-Atlas (WU et al., 2025), ao produzir variantes sobre InternVL-2 (CHEN et al., 2024a) e Qwen2-VL (WANG et al., 2024), evidencia a superioridade do Qwen em ancoramento espacial na ausência de um pré-treino dedicado a esse fim, enquanto agentes de ponta posteriores ao início deste trabalho, como o UI-Venus-1.5 (GAO et al., 2026), retornam ao Qwen3-VL como base.

3.4.2 Conjunto de dados

Adota-se o AgentNet (WANG et al., 2025) como conjunto de dados deste experimento. A base é voltada puramente para ambientes de *desktop*, abrangendo três sistemas operacionais distintos: Windows, Ubuntu e macOS. Ela contém trajetórias completas para a execução de tarefas com diferentes níveis de complexidade, incluindo tarefas impossíveis ou com obstáculos propositais. Rotas com erros ou passos redundantes também são incluídas, mesmo em casos de falha da tarefa. Todas as amostras são coletadas por demonstração humana e rotuladas de forma abrangente com auxílio de modelos multimodais.

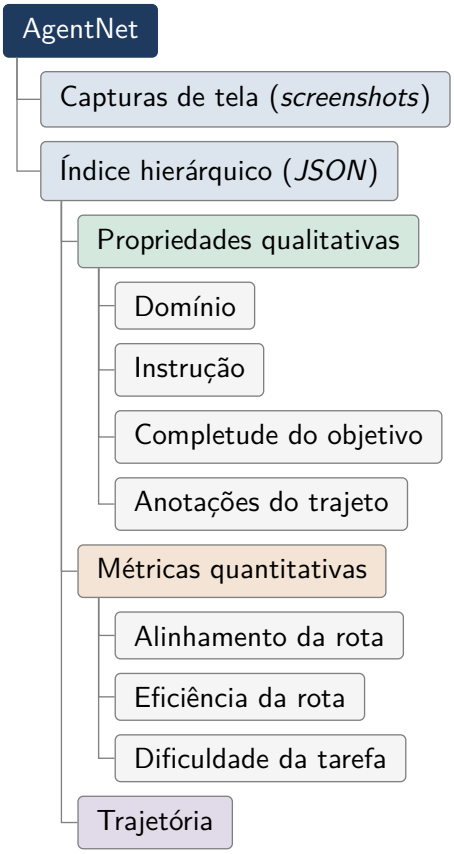


Figura 7 – Organização estrutural do conjunto de dados AgentNet.

O *dataset* organiza-se em uma coleção de capturas de tela e um índice hierárquico em formato *JSON*, como ilustra a Figura 7. O arquivo armazena propriedades qualitativas de cada tarefa, incluindo seu domínio, a instrução a ser seguida, a completude do objetivo e anotações sobre o trajeto, como o motivo para uma falha ou o resultado esperado no caso de tarefas com obstáculos propositais. Também se incorporam métricas quantitativas sobre as trajetórias, capturando os graus de alinhamento e eficiência da rota em relação ao objetivo, assim como a dificuldade da tarefa.

Os percursos realizados são representados como listas de passos, contendo uma referência à captura de tela do estado atual e um conjunto próprio de atributos. Dentre esses está a sequência de ações feitas na etapa escolhida, descritas em formato de código conforme a sintaxe da biblioteca `pyautogui`: um pacote popular para a automação de *desktops*. Adicionalmente, o escopo abarca dois indicadores que sinalizam se a última ação executada foi incorreta ou redundante. Completando os metadados, incluem-se ainda quatro anotações textuais geradas por VLMs, compreendendo uma observação do estado atual, uma descrição das ações executadas nesse estado, uma reflexão sobre as ações tomadas e seus resultados e um raciocínio em primeira pessoa sobre o que fazer.

A escolha pelo AgentNet decorre da presença de atributos que tornam a base de dados altamente aplicável para os objetivos da pesquisa: o domínio de *desktop*, a disponibilidade de trajetórias sequenciais completas, permitindo o uso do regime *off-policy* com histórico por passo, e a coleta por demonstração humana. As alternativas existentes não possuem as mesmas vantagens. As principais bases de trajetórias concentram-se em ambientes móveis e *web*, como o AndroidControl (LI et al., 2024), o GUI-Odyssey (LU et al., 2025a) e o Mind2Web (DENG et al., 2023). Opções voltadas a *desktop* limitam-se ao *grounding* estático de elementos, como no OS-Atlas (WU et al., 2025), ou a amostras derivadas de uma única captura de tela sem estados intermediários, como no OmniACT (KAPOOR et al., 2024). Esse também é o caso em *datasets* agregados ou universais com fatias voltadas para *desktop*, como no GUI-R1 (LUO et al., 2025). No caso de *pipelines* de síntese automática, como no OS-Genesis (SUN et al., 2025), escala-se a coleta ao custo de um ruído residual não rotulado.

O AgentNet ainda incorpora passos incorretos ou redundantes e tarefas impossíveis acompanhados de indicadores explícitos, atributo raro entre as demais bases, que amplia a diversidade da distribuição de treino e viabiliza o controle fino da curadoria.

3.4.3 Regime de treino e enquadramento temporal

A otimização do modelo baseia-se no aprendizado por reforço sob o regime *off-policy*, em que a "política" que gera a distribuição de treino difere daquela que está sendo ajustada. No contexto de CUAs, isso geralmente implica o uso de um conjunto de dados

pré-amostrado. Técnicas *on-policy* modernas para agentes usuários de GUIs baseiam-se em máquinas virtuais ou ambientes simulados para interação em tempo real (BAI et al., 2024; XIE et al., 2024). Embora ofereçam exploração dinâmica e adaptação contínua, essas estratégias impõem custos computacionais e temporais proibitivos, uma vez que a exploração livre gera um alto volume de erros iniciais que prolonga significativamente a convergência do modelo. Frente à premissa do uso de métodos compatíveis com *hardware* comercial, um regime baseado em dados pré-amostrados torna-se a alternativa mais viável para o treinamento.

O enquadramento temporal dos dados define como as trajetórias pré-amostradas alimentam o modelo e como a recompensa é atribuída ao longo delas. Inspirando-se em Luo et al. (2025), adotam-se episódios de um passo, nos quais a recompensa incide sobre uma única captura de tela e as ações nela previstas, em vez de sobre a trajetória inteira. A formulação beneficia a eficiência no uso dos dados. O desacoplamento temporal das ações maximiza o aproveitamento do *dataset* pré-amostrado, pois isolar os passos permite extrair sinais de aprendizado válidos de qualquer ponto da trajetória, de modo que falhas ou ações redundantes no início de uma demonstração não comprometem a utilidade das decisões corretas tomadas em sequência.

O mesmo isolamento beneficia a estabilidade da otimização. Como cada amostra fornece um histórico pré-registrado, em vez de depender das predições passadas do próprio modelo, decisões ruins não produzem efeitos de cascata. A isso soma-se a independência entre amostras, que viabiliza *batches* de maior variância, combinando diferentes tarefas e ambientes e estabilizando a atualização dos gradientes. A avaliação de cada passo de forma isolada mitiga ainda o problema de atribuição de crédito, recorrente em trajetórias longas de navegação, nas quais é difícil identificar qual ação determinou o sucesso ou o fracasso da tarefa, fazendo o sinal de aprendizado atuar diretamente sobre a ação imediata.

3.4.4 Algoritmo de otimização

A otimização da política do agente emprega o algoritmo GSPO (ZHENG et al., 2025). A dispensa de um modelo crítico operando em paralelo à política reduz drasticamente o consumo de memória de vídeo (VRAM). Essa eficiência algorítmica constitui um fator determinante para viabilizar o treinamento sob restrições de *hardware* comercial. Adicionalmente, a escolha metodológica apoia-se no sucesso de abordagens recentes de otimização para raciocínio e automação, inspirando-se diretamente nos resultados consolidados pelos modelos DeepSeek-R1 (DeepSeek-AI, 2025) e GUI-R1 (LUO et al., 2025).

3.4.5 Espaço de ações

A escolha do espaço de ações usado neste trabalho fundamenta-se no alinhamento estrutural com o *dataset* selecionado. Uma vez que a base de dados utiliza a biblioteca `pyautogui` para a captura e o registro das interações com o ambiente, torna-se coerente que a saída do modelo adote o mesmo padrão. O pacote também é frequentemente aplicado em *benchmarks* para navegação de GUIs, como em Xie et al. (2024). Pode-se subdividir esse vocabulário em quatro categorias funcionais: comandos de ponteiro, rolagem de tela, entradas de teclado e controle de fluxo.

Para garantir invariância à resolução das capturas de tela, as coordenadas espaciais (x, y) das ações de ponteiro, como `click` e `dragTo`, são contínuas e estritamente normalizadas no intervalo $[0, 1]$. Em contraste, as rolagens de *mouse* operam via deslocamentos relativos representados por valores inteiros. Comandos de teclado processam tanto *strings* quanto sequências estruturadas de teclas ou atalhos. Já ações de controle fornecem métodos para aguardar mudanças de tela (`wait`) e sinalizar o encerramento da trajetória (`terminate`).

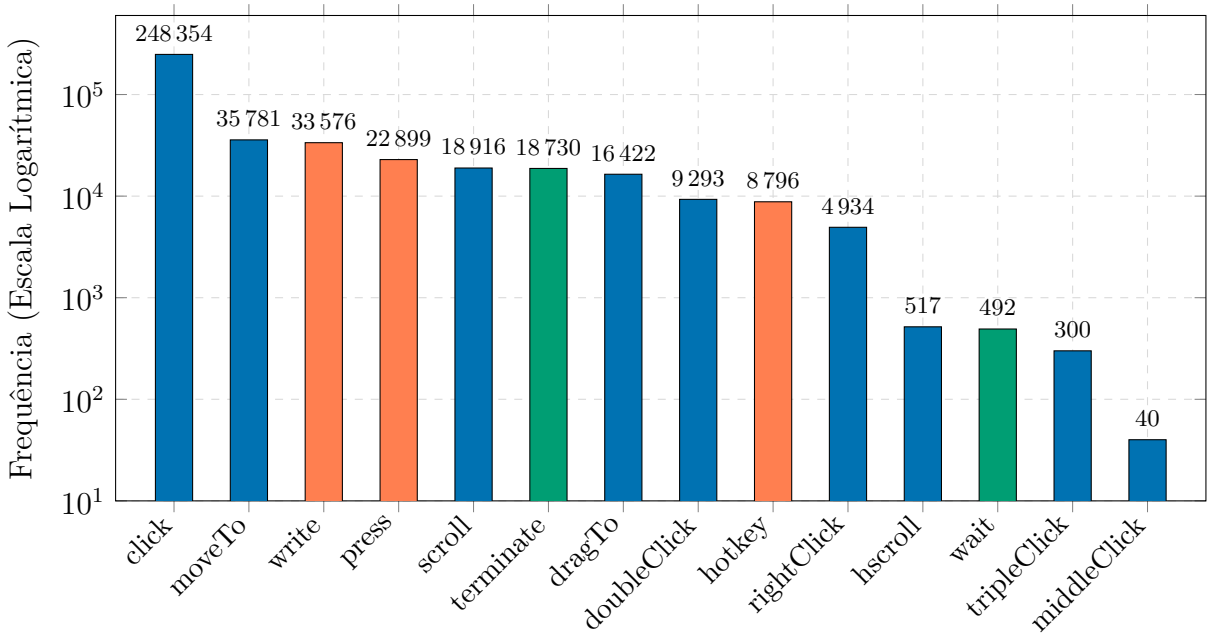


Figura 8 – Distribuição de frequência das ações válidas no *dataset* AgentNet após a filtragem dos dados (Seção 3.5.1). As cores agrupam as ações em comandos de ponteiro e rolagem, entradas de teclado e controle de fluxo.

A distribuição dessas ações ao longo das trajetórias presentes nos dados, apresentada na Figura 8, evidencia um forte desbalanceamento, refletindo padrões orgânicos de navegação. A maior parte das interações concentra-se em comandos como cliques simples, movimentações de ponteiro e inserções textuais. Ações contextuais mais específicas, como rolagens horizontais, cliques sucessivos e uso do botão central do *mouse*, ocorrem de forma

esparsa. Essa assimetria exige que o modelo aprenda uma política abrangente para os comandos majoritários enquanto mantém a habilidade de inferir ações raras. Todas as ações possíveis e seus argumentos estão incluídos no *prompt* do sistema (disponível no Apêndice 6.1).

Uma variante reduzida desse vocabulário, que exclui as operações de *grounding* como cliques e movimentos de ponteiro, é empregada em uma das configurações experimentais, conforme definido na Seção 3.6.

3.4.6 Estrutura das entradas

Para viabilizar o aprendizado em passos independentes, a entrada do modelo deve encapsular todo o contexto necessário para a tomada de uma decisão correta e imediata. A fim de provê-lo a cada etapa, opta-se pelo uso de um *prompt* multimodal em forma de conversa, seguindo o padrão ChatML (OpenAI, 2023), que organiza o texto em mensagens demarcadas por papéis explícitos: sistema, usuário e assistente. Seguindo essa demarcação, estrutura-se a entrada contendo a instrução da tarefa, o estado visual atual e o histórico pré-definido de ações anteriores.

A primeira mensagem recebida pelo modelo é o *prompt* do sistema, que descreve como o agente de navegação deve se comportar. Baseado em Luo et al. (2025) e Wang et al. (2025), ele especifica o propósito da rede, o conteúdo e formato do histórico recebido, o espaço de ações disponível e o padrão de saída desejado a fim de permitir o uso posterior de ferramentas de análise sintática. A diretriz ainda inclui orientações sobre como raciocinar a respeito do problema e alguns exemplos de saída a fim de induzir um melhor entendimento. O texto completo está disponível no Apêndice 6.1.

Em seguida, estrutura-se o histórico de navegação intercalando mensagens de usuário e assistente, conforme uma combinação das formulações L1 e L3 propostas por Wang et al. (2025). Para otimizar o uso de recursos, adota-se uma representação com nível de detalhamento decrescente. O passo imediatamente anterior retém a captura de tela, acompanhada de sua descrição textual e das ações executadas. Para os dois passos precedentes a este, mantêm-se apenas a observação textual do estado e a respectiva ação. Por fim, para os registros mais antigos, preservam-se exclusivamente as ações tomadas, reproduzindo a configuração metodológica estabelecida em Luo et al. (2025). Nesse arranjo, as capturas de tela compõem as mensagens do usuário, enquanto as ações e observações prévias são atribuídas ao papel do assistente. A Figura 9 ilustra essa organização.

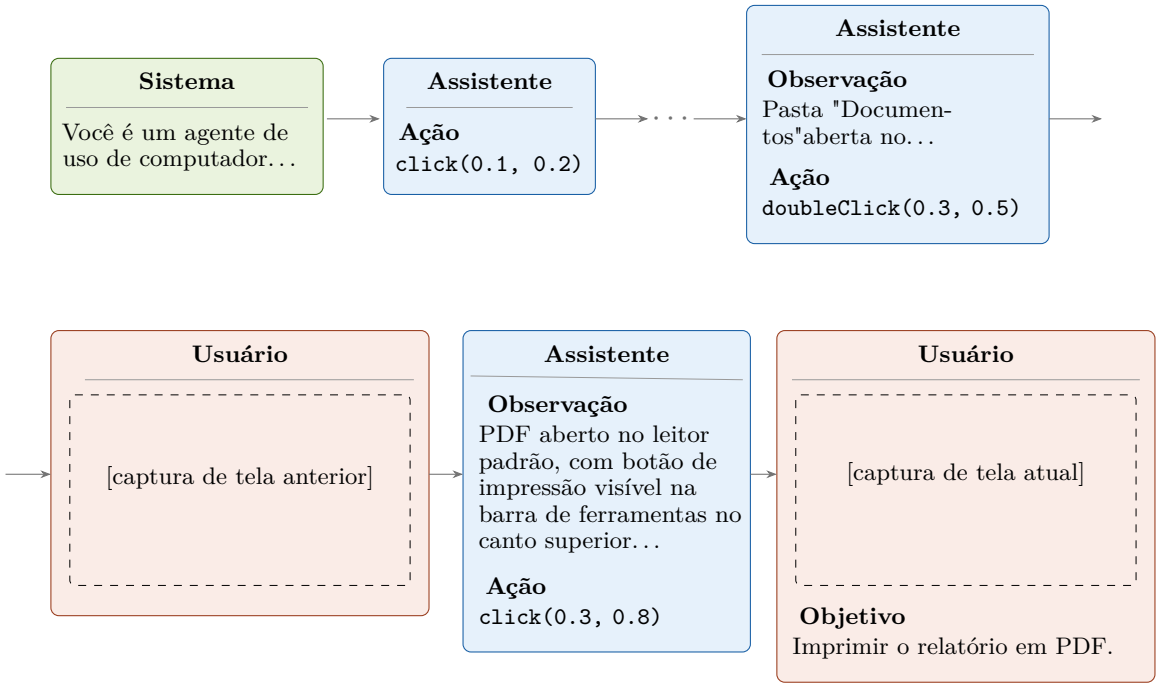


Figura 9 – Estrutura da entrada fornecida ao modelo durante a inferência. A mensagem do sistema é apresentada em verde, as do assistente em azul e as do usuário em vermelho. A primeira linha representa o histórico até a penúltima etapa, enquanto a segunda linha mostra o passo imediatamente anterior, que mistura mensagens de assistente e usuário. Por fim, inclui-se a captura atual acompanhada do objetivo.

Por último, o modelo recebe a captura de tela atual acompanhada do objetivo a ser seguido como uma mensagem de usuário. Opta-se por sempre incluir o objetivo na mensagem mais recente para reforçar a atenção da rede sobre a meta primária, mitigando o fenômeno de diluição de contexto que frequentemente ocorre após o processamento de históricos extensos. Assim, garante-se que a predição da próxima ação permaneça estritamente condicionada à diretriz principal da tarefa.

3.5 Preparação dos dados de treino

Esta seção descreve a construção das amostras de treino a partir das trajetórias brutas do AgentNet, em duas etapas: a filtragem e curadoria das trajetórias e a padronização das ações válidas em rótulos, que se combinam à estrutura de entrada da Seção 3.4.6 para compor as amostras finais de treino.

3.5.1 Filtragem e curadoria das trajetórias

A extração de rótulos de treinamento de alta qualidade é um pré-requisito crítico para a obtenção de sinais de aprendizado consistentes. Os comportamentos orgânicos presentes no AgentNet, descritos na Seção 3.4.2, fornecem trajetórias densas e realistas,

mas incluem navegações ruidosas, passos redundantes e tarefas abandonadas. A fim de garantir que as ações utilizadas como referência reflitam estritamente escolhas úteis e coerentes com a resolução do objetivo, estabelece-se um processo de curadoria que descarta transições prejudiciais e consolida os rótulos definitivos de treinamento.

O processo de filtragem opera em dois escopos complementares, atuando em nível de tarefa, ou episódio, e em nível de passos individuais. Inicialmente, as tarefas são rejeitadas com base em sua completude e pontuação de alinhamento. Trajetos concluídos com sucesso são integralmente preservados. Tarefas incompletas passam ainda por uma segunda triagem a fim de preservar passos intermediários contendo demonstrações válidas de navegação. Nesses casos, a métrica de alinhamento incluída nos dados é utilizada como critério de corte, garantindo que as ações executadas possuam estrita coerência com o objetivo da tarefa.

Ainda no contexto de episódios incompletos, aplicam-se duas restrições adicionais para proteger o sinal de treino. Primeiramente, tarefas contendo instruções demasiadamente curtas são removidas para evitar ambiguidades. Além disso, a última ação registrada, invariavelmente uma chamada de encerramento (`terminate`), é removida. Sua exclusão impede que o agente aprenda a sinalizar o fim da tarefa de forma precoce ao se deparar com cenários de falha.

Para passos individuais, a avaliação é feita por meio dos atributos de redundância e incorreção fornecidos pelo *dataset*. Etapas rotuladas como redundantes ou incorretas são descartadas. Passos válidos subsequentes são mantidos, mesmo que a etapa imediatamente anterior tenha sido removida. Assim, garante-se o aproveitamento máximo das demonstrações corretas disponíveis.

3.5.2 Padronização das ações e geração dos rótulos

Após a filtragem, padronizam-se as ações válidas para a geração dos rótulos definitivos. Originalmente, a base de dados codifica as interações como instruções da biblioteca `pyautogui`. Para adequá-las ao formato de validação do treinamento, empregam-se expressões regulares (*regex*) e rotinas de *parsing* textual. Esse processamento remove as invocações de classe e converte blocos de múltiplas linhas em uma lista de dicionários, na qual o nome do comando e seus respectivos argumentos são mapeados de forma independente. Como exemplo, o trecho de código:

```
pyautogui.scroll(-15)
pyautogui.click(x=0.3, y=0.6)
pyautogui.write(message="hello")
```

é estruturado internamente na seguinte representação *JSON*:

```
[
    {"action": "scroll", "arg_0": -15},
    {"action": "click", "x": 0.3, "y": 0.6},
    {"action": "write", "message": "hello"}
]
```

Durante o treinamento, essa estrutura atua como o alvo direto de comparação. Cada amostra reúne um passo preservado pela curadoria, montado como entrada segundo a estrutura de conversa da Seção 3.4.6, e um rótulo formado pela lista padronizada das ações restantes do episódio a partir daquele passo. As respostas geradas pelo modelo durante a inferência são similarmente decompostas por meio de *regex*, viabilizando o pareamento de cada ação prevista com esse rótulo.

3.6 Configurações experimentais e execução

Definidos os fatores fixos e as amostras que alimentam o treino, resta especificar sob que configurações o experimento foi executado. O estudo compreende duas configurações de treinamento. A configuração preliminar, de caráter exploratório, opera sobre a variante reduzida do espaço de ações (Seção 3.4.5) e presta-se exclusivamente ao esclarecimento do comportamento do modelo ao longo do treino. A configuração principal incorpora o espaço de ações integral e produz o modelo submetido às respostas finais.

A Tabela 2 consolida as variáveis do experimento, distinguindo as fixas por projeto, as com variação entre configurações e as não controladas. As fixas por projeto correspondem às decisões descritas na Seção 3.4. Os nove fatores com variação assumem níveis distintos entre as duas configurações e são detalhados adiante. As não controladas não são manipuladas pelo experimentador, sendo registradas de forma factual e discutidas na Seção 4.6.

Tabela 2 – Classificação das variáveis do experimento.

Grupo	**Variáveis**
Fixas por projeto	Modelo base (Qwen3-VL) Conjunto de dados (AgentNet) Regime *off-policy* com episódios de um passo Algoritmo de otimização (GSPO) Formulação da recompensa, pesos (w_1, w_2) e tolerâncias de validação Definição completa do espaço de ações Estrutura das entradas Número de gerações por amostra (4)
Com variação entre configurações	Variante do espaço de ações *Batch* efetivo Acumulação de gradiente Taxa de aprendizado máxima Razão de *warmup* Limite máximo de sequência *Hardware* Fração do conjunto de dados processada Duração da execução
Não controladas	Semente e estocasticidade da geração Ordem e subconjunto dos dados efetivamente vistos Não-determinismo do ambiente dinâmico na avaliação OSWorld-Verified Condições da infraestrutura de nuvem

Os níveis assumidos pelos fatores com variação em cada configuração estão registrados na Tabela 3. Cabe observar que o *batch* efetivo resulta da combinação do *batch* por dispositivo, da acumulação de gradiente e do número de GPUs, razão pela qual esses três elementos são registrados separadamente.

Tabela 3 – Matriz experimental: níveis dos fatores em cada configuração.

Fator	**Preliminar**	**Principal**
Espaço de ações	Sem *grounding*	Completo
Hardware	1 × RTX 4080 (16 GB)	8 × RTX PRO 6000 (96 GB)
Batch por dispositivo	1	8
Acumulação de gradiente	4	1
Batch efetivo	4	64
Taxa de aprendizado máxima	5×10^{-6}	4×10^{-5}
Razão de *warmup*	0,1	0,02
Limite de sequência (*tokens*)	1024	512
Fração do *dataset* processada	∼50%	∼10%
Duração da execução	∼16 dias	∼16 horas

Três fatores de otimização foram ajustados entre as configurações. A taxa de

aprendizado máxima foi elevada com o intuito de acelerar a convergência, dado o menor número de passos de otimização decorrente do *batch* ampliado. A razão de *warmup* foi reduzida de modo a posicionar o fim do aquecimento em torno da mesma quantidade de amostras da configuração preliminar, compensando o crescimento do conjunto de dados após a inclusão das ações de *grounding*. A acumulação de gradiente foi dispensada, uma vez que a VRAM disponível na configuração principal comporta todas as gerações do GSPO em um único passo. Já o limite de sequência foi reduzido com base na análise dos comprimentos de geração observados na configuração preliminar.

As duas configurações foram executadas em ambientes computacionais distintos. A preliminar rodou localmente, em uma única placa de consumo cedida pelo laboratório Model2AI do Departamento de Engenharia de Computação e Automação da Universidade Federal do Rio Grande do Norte. A principal recorreu a uma infraestrutura de servidor em nuvem, alugada por meio da plataforma de locação de GPUs Vast.ai. Mesmo nesse ambiente, o número de gerações por amostra e o limite de sequência foram mantidos em valores compatíveis com *hardware* comercial, restringindo-se o uso do servidor à ampliação do *batch* para acelerar o treino. Cada configuração foi executada uma única vez, sem replicações com sementes distintas.

3.7 Plano de análise

A análise dos resultados, apresentada no capítulo seguinte, organiza-se em duas frentes correspondentes aos níveis de resposta definidos na Seção 3.2. A primeira examina as respostas de processo ao longo das duas execuções, com finalidade diagnóstica sobre o comportamento e a estabilidade da otimização. A segunda examina as respostas finais, comparando o modelo ajustado ao modelo base em cada *benchmark*, com subdivisões por categoria quando aplicável.

As duas frentes são apresentadas e interpretadas separadamente. A relação entre elas, isto é, se o progresso na grandeza otimizada se traduz em melhora nas métricas de avaliação, é tratada como objeto de análise. A leitura dos resultados é acompanhada da discussão de ameaças à validade (Seção 4.6), parte integrante de sua interpretação.

4 RESULTADOS

A avaliação do agente proposto organiza-se em torno de duas frentes complementares. A primeira examina o desempenho externo do modelo, medindo sua competência em *benchmarks* consolidados de interação com interfaces gráficas. A segunda foca nas métricas internas registradas ao longo do treino, investigando a dinâmica do procedimento de ajuste e os fatores influentes sobre o sinal de aprendizado. Em conjunto, os dois estudos permitem não apenas quantificar a capacidade da rede resultante, mas também diagnosticar os gargalos e as oportunidades inerentes à abordagem adotada.

As avaliações de desempenho recaem sobre o treino principal definido na Seção 3.6. Por corresponder ao processamento de apenas cerca de 10% do conjunto de dados, o *checkpoint* avaliado reflete tendências em formação, e não um modelo plenamente convergido. Sua *performance* é interpretada com o auxílio do treino preliminar que, embora não comporte avaliação direta nos *benchmarks*, reequilibra a distribuição de ações e estende o horizonte de treino viável, oferecendo uma janela complementar sobre a dinâmica de treinamento de longo prazo, explorada ao longo deste capítulo.

4.1 Protocolo de avaliação

O desempenho do modelo é aferido em dois *benchmarks* que isolam capacidades distintas. O *ScreenSpot-Pro* (LI et al., 2025) mede o ancoramento espacial puro, exigindo a localização de um único elemento em uma tela de alta resolução a partir de uma instrução textual. Já o *OSWorld-Verified* (XIE et al., 2024) avalia a competência do agente de ponta a ponta, inserindo o modelo em um ambiente executável onde deve completar tarefas reais por meio de múltiplas interações sequenciais. A recompensa para cada tarefa é binária, sendo definida pelo sucesso ou fracasso do objetivo. Por meio deste, valida-se a execução de tarefas em horizonte longo.

Para a avaliação realizada no *OSWorld-Verified*, emprega-se um subconjunto de 361 tarefas distribuídas em dez domínios. Tanto o modelo base quanto o treinado operam sob condições idênticas: espaço de ações `pyautogui`, observação por captura de tela, limite de quinze passos por tarefa, temperatura nula e janela de histórico de doze passos. O modelo base corresponde ao Qwen3-VL de 4 bilhões de parâmetros, sem qualquer ajuste, operando sob o mesmo *prompt* e agindo como referencial para isolar o efeito da metodologia adotada.

4.2 Desempenho em ancoramento espacial

No *ScreenSpot-Pro*, o ajuste produziu um ganho consistente sobre o modelo base. A acurácia geral evoluiu de 33,7% para 38,7%, um avanço de cinco pontos percentuais. A decomposição por tipo de alvo, apresentada na Tabela 4 e ilustrada na Figura 10, revela que esse progresso concentra-se na localização de elementos textuais, cuja acurácia subiu de 46,7% para 54,15%. A precisão sobre ícones, categoria reconhecidamente mais difícil por depender de pistas puramente visuais e desprovidas de rótulo, manteve-se praticamente estável, com leve elevação de 12,7% para 13,7%.

Tabela 4 – Acurácia no *benchmark ScreenSpot-Pro*, por tipo de alvo.

Modelo	**Texto (%)**	**Ícone (%)**	**Geral (%)**
Base (Qwen3-VL 4B)	46,70	12,70	33,70
Treinado	54,15	13,70	38,70
Δ (%)	$+7,45$	$+1,00$	$+5,00$

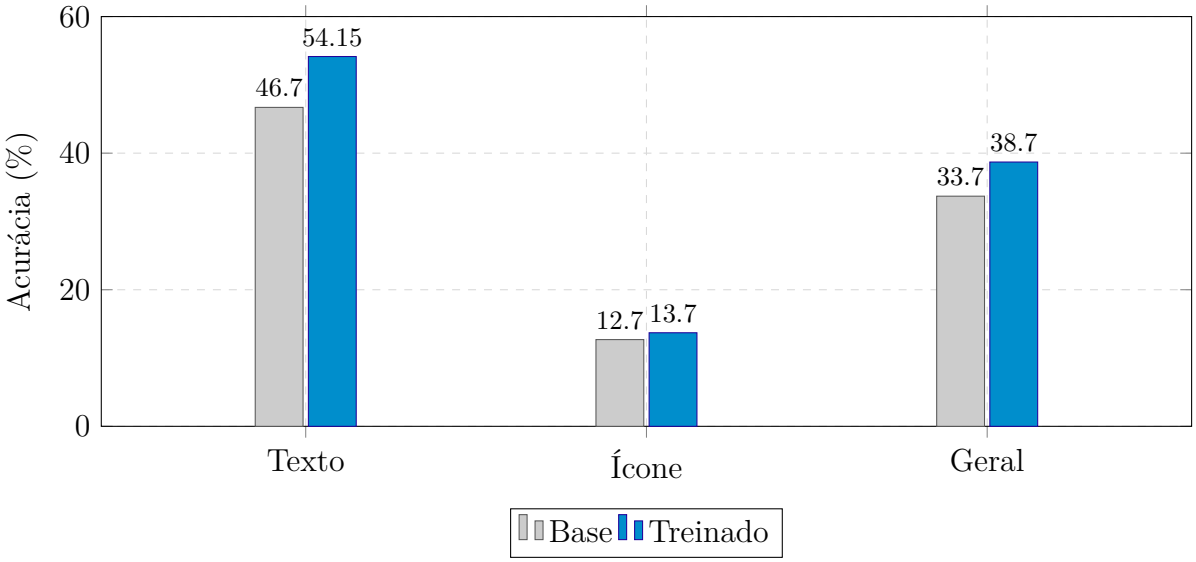


Figura 10 – Comparação da acurácia no *ScreenSpot-Pro* entre o modelo base e o treinado.

O resultado indica que o procedimento de ajuste refinou a capacidade de correlacionar instruções textuais a regiões da tela, transferindo esse aprimoramento para uma distribuição de imagens distinta da observada em treino. Cabe destacar que esse ganho emerge sem qualquer objetivo explícito dedicado ao ancoramento. Não há uma fase de treino dedicada ao *grounding*, e a única pressão sobre a precisão das coordenadas advém do raio curto de tolerância exigido para que um clique seja considerado correto (Seção 3.5.1). A localização espacial é aprendida de forma implícita, como subproduto da exigência de acerto da ação. O fato de um aprimoramento mensurável surgir mesmo nessas condições sugere que otimizar o ancoramento de modo conjunto com as demais competências não é necessariamente contraproducente, ao menos sobre uma base como o Qwen3-VL, que

já dispõe de fundação sólida em localização espacial. O ganho assimétrico em favor do texto indica que essa pressão atuou de forma mais efetiva sobre alvos cuja identificação se beneficia da leitura de rótulos, capacidade já enfatizada no pré-treino da família Qwen, enquanto o ancoramento de ícones permanece como desafio em aberto.

4.3 Desempenho em tarefas agênticas

O comportamento observado no *OSWorld-Verified* contrasta o ganho em ancoramento. O modelo treinado obteve 11,31% de sucesso agregado: uma queda de 3,8 em relação aos 15,10% do modelo base. A Tabela 5 discrimina esse resultado por domínio, computando as pontuações parciais conforme as tarefas do *benchmark*.

Tabela 5 – Taxa de sucesso no *OSWorld-Verified* por domínio (crédito parcial).

Domínio	**Tarefas**	**Base (%)**	**Treinado (%)**	**Δ (%)**
chrome	46	23,83	23,91	+0,09
gimp	26	30,77	15,38	−15,38
libreoffice_calc	47	6,38	2,13	−4,26
libreoffice_impress	47	12,56	10,64	−1,92
libreoffice_writer	23	13,03	4,34	−8,70
multi_apps	93	9,31	4,11	−5,21
os	24	20,83	29,17	+8,33
thunderbird	15	33,33	26,67	−6,67
vlc	17	11,76	5,88	−5,88
vs_code	23	13,04	13,04	0,00
Geral	**361**	**15,10**	**11,31**	−3,80

Contrariamente ao cenário ilustrado pela leitura direta da comparação, uma análise por tarefa revela que os dois modelos concordam em 323 das 361 tarefas (89%), com 29 resolvidas por ambos e 294 falhadas por ambos. A divergência concentra-se em 38 tarefas. O modelo treinado avança no domínio `os` (+8,3%) e mantém performance comparativa em `chrome` e `vs_code`, abrangendo categorias alinhadas ao perfil de navegação enfatizado no *dataset*, enquanto regride nos domínios de aplicações gráficas de produtividade, como visto para o *GIMP*.

4.3.1 Análise comportamental

A divergência observada não reside na incapacidade de localizar ou operar elementos, mas no encerramento das tarefas. A Tabela 6 reúne indicadores comportamentais agregados sobre as 361 tarefas. O modelo treinado encerra com sucesso menos vezes que o base, ultrapassa o limite de passos com maior frequência e entra em laços repetitivos de ação em maior proporção.

Tabela 6 – Indicadores comportamentais agregados no *OSWorld-Verified*.

Indicador	**Base**	**Treinado**
Passos médios por tarefa	13,5	14,1
Encerrou com sucesso (%)	15	10
Esgotou o limite de passos (%)	84	89
Entrou em laço repetitivo ($\geq 3\times$) (%)	68	75

Nas tarefas em que os modelos divergem, o fator determinante é o reconhecimento da conclusão. Ambos produzem prefixos de ações corretos e, com frequência, idênticos no tipo e na sequência das ações, ainda que com pequenas diferenças de coordenadas, indicando que a habilidade de ancoramento permanece preservada. A diferença surge após o objetivo ser atingido: o modelo base interrompe a execução, enquanto o treinado prossegue, reabrindo menus e reaplicando operações que comprometem um resultado correto. A Figura 11 ilustra esse mecanismo em uma tarefa do *GIMP*, na qual ambos os modelos alcançam o estado-alvo com a mesma sequência inicial, mas apenas o base reconhece o término.

```
BASE (7 passos):                TREINADO (14 passos):
 1. click(1056,691)              1. click(1056,691)
 2. click(250,76)                2. click(269,76)
 3. click(288,140)               3. click(288,140)
 4. click(288,140)               4. click(288,140)
 5. click(557,184)               5. click(557,184)
 6. click(1171,799)              6. click(1152,799)  <- objetivo atingido
 7. terminate                    7. click(269,76)    <- reabre o menu
                                  8-13. ...repete a operacao 2 vezes...
                                 14. terminate
```

Figura 11 – Comparação de trajetórias em uma tarefa do *GIMP*. Ambos atingem o objetivo com o mesmo prefixo de ações; o modelo treinado prossegue além do ponto de conclusão e invalida o resultado.

Constata-se, assim, que a regressão agregada decorre de uma falha comportamental localizada e não de uma perda de capacidade perceptiva ou de *grounding*. Esse achado é coerente com o ganho observado no *ScreenSpot-Pro*: a competência de localização não apenas foi preservada, como melhorou, ao passo que a habilidade de horizonte longo de reconhecer a tarefa como concluída deixou de ser adequadamente induzida pelo treino.

4.4 Métricas internas de treinamento

O monitoramento das recompensas ao longo do ajuste decompõe-se nos dois componentes definidos no Capítulo 3: a recompensa de formato, limitada a 0,6, e a recompensa de acurácia, que agrega os termos de tipo e acerto da ação e atinge no máximo 0,4.

Acompanha-se ainda o comprimento médio das gerações e a fração de grupos de amostragem com desvio padrão de recompensa nulo, que indica a parcela de *prompts* cujas amostras recebem recompensas idênticas e, portanto, não geram gradiente sob o GSPO.

4.4.1 Saturação do formato e estreiteza do sinal de acurácia

Em ambos os treinos, a recompensa de formato cresce rapidamente até seu teto e ali permanece. No treino principal, ela satura próximo de 0,6 por volta do passo 80. No preliminar, já parte saturada. A Figura 12 apresenta a evolução das recompensas no treino principal e evidencia que, uma vez saturado o formato, a única dimensão com margem de aprendizado é a acurácia, cujo valor oscila em uma faixa estreita. A recompensa total estabiliza-se em torno de 0,74, dominada pela parcela fixa de formato. Esse comportamento é consistente com modelos da família Qwen, que são treinados para a obediência de instruções.

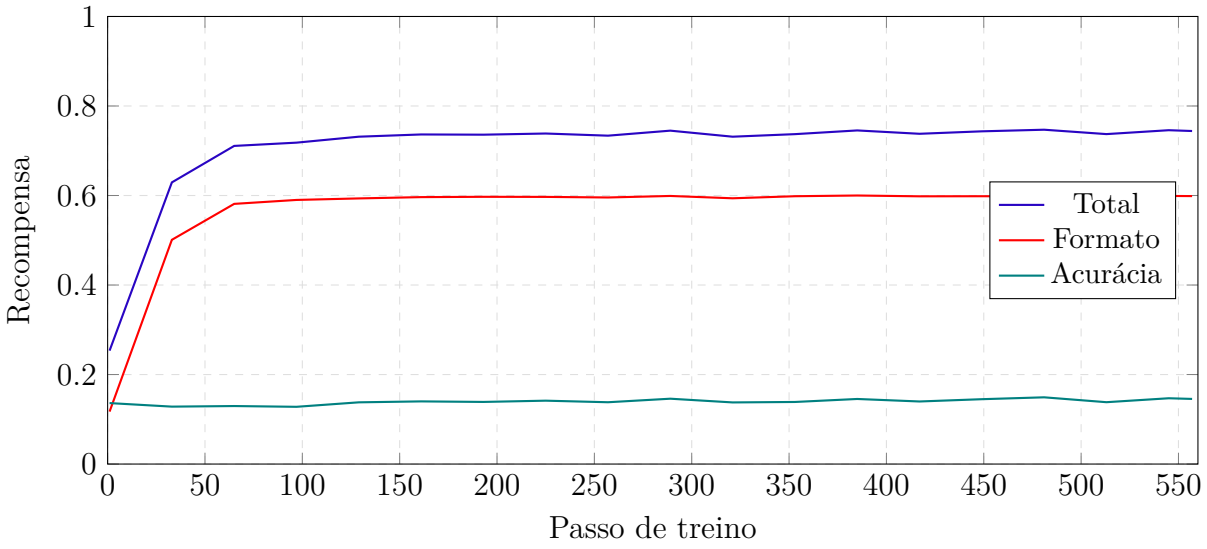


Figura 12 – Evolução das recompensas no treino principal. O formato satura próximo de seu teto de 0,6 nos primeiros passos. A acurácia, único componente com margem, permanece quase plana.

A acurácia é o único componente com margem de evolução, e as duas execuções diferem em seu comportamento. No treino principal, a recompensa de acurácia varia de cerca de 0,127 para 0,145 ao longo dos 557 passos registrados. No treino preliminar, a mesma métrica sobe de forma lenta e contínua, partindo de 0,046 e alcançando aproximadamente 0,17, cerca de 42% de seu teto normalizado. A Figura 13 compara as duas trajetórias em função do número de exemplos processados, e não de passos de otimização, atenuando a diferença no tamanho de *batch* entre os treinos.

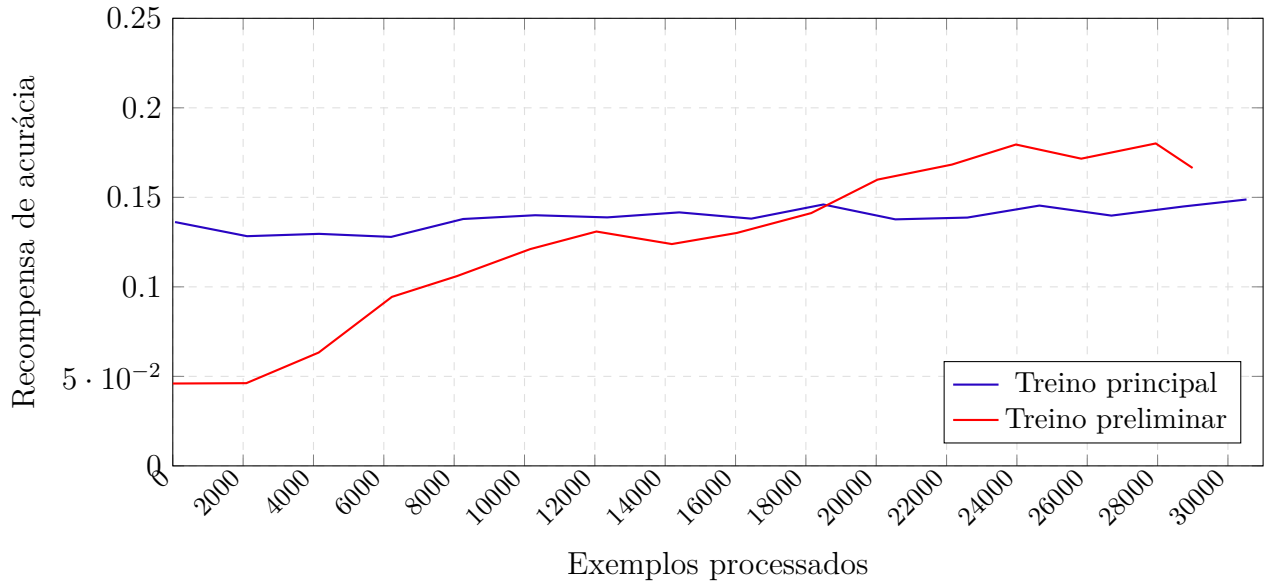
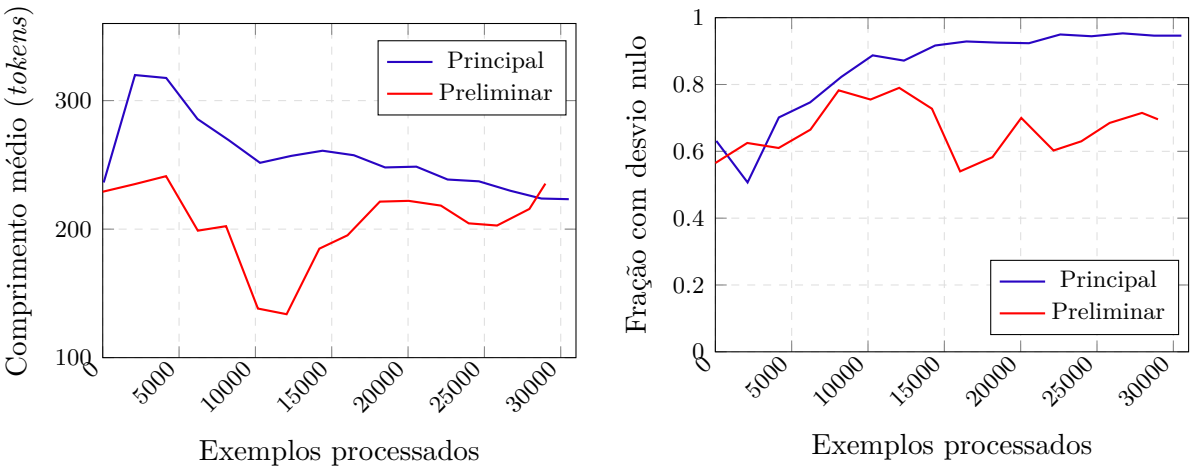


Figura 13 – Recompensa de acurácia em função do número de exemplos processados. O eixo em exemplos atenua a diferença de tamanho de *batch*, mas o treino principal acumulou muito menos atualizações de parâmetros que o preliminar, de modo que a comparação é apenas indicativa.

4.4.2 Comprimento das gerações e variância das amostras

Um padrão adicional emerge da relação entre o comprimento médio das sequências geradas e a variância das recompensas. Em ambos os treinos, observa-se uma associação positiva entre as duas grandezas: gerações mais longas acompanham maior dispersão de recompensa entre as amostras de um grupo, ao passo que, à medida que as sequências encurtam, a fração de grupos com desvio nulo se eleva. No treino principal, esse acoplamento é nítido, com correlação de aproximadamente $+0,65$ entre o comprimento das gerações e a variância das recompensas. Ao longo do ajuste, o comprimento médio das gerações declina de cerca de 250 para 223 *tokens*, movimento que coincide com a elevação da fração de desvio nulo apresentada nesta subseção. O encurtamento progressivo das respostas, portanto, contribui para a erosão do sinal de aprendizado, reduzindo a variabilidade sobre a qual a vantagem relativa ao grupo é computada.

Observa-se ainda que, em nenhum dos treinos, o comprimento médio das gerações se aproxima dos respectivos limites de sequência. No treino preliminar, configurado com limite de 1024 *tokens*, as gerações situam-se em torno de 200 *tokens*. No principal, com limite reduzido para 512 *tokens*, a média oscila entre 220 e 250 e tende a diminuir. A folga consistente em relação ao teto confirma que a redução do limite de sequência, decidida a partir das métricas do treino preliminar, foi adequada e não introduziu truncamento relevante.



(a) Comprimento das gerações

(b) Fração com desvio padrão nulo

Figura 14 – Comprimento médio das gerações (a) e fração de grupos com desvio padrão de recompensa nulo (b), em função do número de exemplos processados, para os dois treinos.

4.5 Dinâmica de terminação

A falha de sub-terminação identificada na avaliação agêntica motiva uma investigação específica sobre a capacidade do modelo de aprender a encerrar tarefas. Sob a formulação de passo único adotada neste trabalho, a ação `terminate` é o alvo correto em apenas cerca de 5% dos estados do treino principal, valor que decorre diretamente da geometria das trajetórias: cada percurso contém uma única ação de encerramento em meio a aproximadamente duas dezenas de outras ações. Essa proporção estrutural induz um forte viés de continuidade, ensinando o modelo a quase nunca parar.

O treino preliminar oferece, nesse ponto, um contraste instrutivo. A exclusão das ações de ancoramento de seus alvos reduz o denominador de ações e eleva a representação de `terminate` para aproximadamente 30% dos alvos, cerca de seis vezes mais que no treino principal. Esse reequilíbrio configura uma espécie de experimento natural, permitindo observar o comportamento de terminação quando a ação dispõe de representação adequada. Convém ressaltar que essas observações derivam de predições sobre estados de trajetórias especializadas registradas no *log* de treino, e não de execuções agênticas. Medem, portanto, a capacidade de decidir o encerramento sobre quadros pré-definidos, não o reconhecimento de uma conclusão alcançada pelo próprio modelo.

A Tabela 7 reporta a calibração da ação ao longo da fase avançada do treino preliminar. A frequência do alvo permanece estacionária em torno de 30% por toda a janela, de modo que a evolução observada reflete o amadurecimento do modelo, e não uma mudança na distribuição dos dados. Inicialmente, mesmo já avançado o treino, o modelo ainda sub-aciona a terminação, disparando-a em apenas 11,2% dos casos. A partir desse ponto, a métrica calibra-se e estabiliza: por aproximadamente treze mil passos, o modelo

Tabela 7 – Calibração da ação `terminate` no treino preliminar, por faixa de passos (fase avançada do treino).

Faixa (passo)	**Alvo term. (%)**	**Modelo term. (%)**	***Recall* (%)**	**Prec. (%)**
13565–16155	30,4	11,2	32,3	87,9
16160–18750	30,3	26,8	69,4	78,4
18755–21345	30,4	29,9	72,8	74,2
21350–23940	32,8	31,8	71,8	73,9
23945–26535	32,0	31,0	72,3	74,5
26540–29125	28,8	28,0	70,5	72,4

aciona `terminate` próximo à taxa-base e sustenta um patamar de *recall* em torno de 71% e precisão em torno de 73%. Trata-se de um platô estável e duradouro, em oposição direta ao comportamento degenerado de quase nunca terminar exibido pelo modelo do treino principal em avaliação.

A evidência indica que a arquitetura é plenamente capaz de desenvolver uma política de terminação calibrada quando a ação correspondente recebe representação suficiente. Soma-se a isso o aspecto de exposição absoluta: no ponto de verificação avaliado, o treino principal havia encontrado da ordem de mil e quinhentos alvos de terminação, cerca de três vezes menos do que o treino preliminar necessitou apenas para iniciar sua calibração.

4.6 Ameaças à validade

A interpretação dos resultados deste capítulo está condicionada às escolhas de projeto e às condições de execução do experimento. As ameaças reunidas a seguir concentram-se nas situações em que essas condições comprometem a correspondência entre os números medidos e as conclusões deles extraídas. Todas têm origem nas variáveis registradas como não controladas ou com variação entre configurações na Tabela 2.

4.6.1 Fatores não isolados entre as configurações

O experimento difere simultaneamente em nove fatores (Tabela 3), enquanto possui apenas duas execuções no total. Não existem ensaios suficientes para sustentar qualquer análise estatística de causa e efeito. Com tantos fatores mudando ao mesmo tempo e tão poucas realizações, não há como isolar o efeito de nenhum fator individual. Qualquer diferença de comportamento ou de resultado entre os treinos pode decorrer de um único fator, da ação combinada de vários ou de interações entre eles, sem meio de distinguir essas possibilidades. Separar essas causas exigiria variar os fatores de forma controlada ao longo de muitas execuções. Dados os recursos disponíveis, conduzir esse número de ensaios seria proibitivo ao cronograma deste trabalho.

Em consequência, o treino preliminar serve apenas para observar como a otimização evolui ao longo do tempo, e não para medir o efeito de qualquer diferença específica em relação ao treino principal. A comparação que mais se aproxima de uma relação de causa e efeito é a das respostas finais, na qual o modelo ajustado e a rede base são avaliados sob condições idênticas (Seção 4.1), o que atribui qualquer diferença observada ao efeito do ajuste, ainda que a medição das respostas finais tenha limitações próprias (Seção 4.6.3).

4.6.2 Execução única sem replicação

Cada configuração foi treinada uma única vez, sem replicações com sementes distintas (Seção 3.6). A semente e a estocasticidade da geração, assim como a ordem e o subconjunto dos dados efetivamente vistos, figuram entre as variáveis não controladas da Tabela 2, e a ausência de réplicas impede estimar sua influência sobre os resultados. Sem repetições, não há como aplicar testes de significância nem construir intervalos de confiança em torno das métricas reportadas, o que restringe ainda mais a validade das conclusões quantitativas. Uma vez que cada resultado provém de um único ajuste, as diferenças medidas nas respostas finais devem ser lidas apenas como evidência indicativa sobre o desempenho das configurações testadas, e não como efeitos já separados do ruído de treino.

4.6.3 Variância de medição no OSWorld-Verified

O OSWorld-Verified instancia ambientes dinâmicos sujeitos a não-determinismo de *timing*, latências e estados transientes. Para uma mesma tarefa e um mesmo modelo, execuções distintas podem divergir, e a avaliação em passagem única não permite quantificar esse erro, que só a repetição da medição permitiria estimar. O ScreenSpot-Pro, por operar sobre capturas fixas, produz uma medição determinística e não compartilha dessa limitação (Seção 3.2.1). Em termos da análise do sistema de medição proposta por Antony (2023), trata-se de uma limitação de repetibilidade do instrumento de avaliação, concentrada sobre as respostas de tarefas agênticas.

O caráter seletivo dessa ameaça é relevante para a leitura dos resultados. O ganho reportado em ancoramento espacial (Seção 4.2) não está sujeito a essa fonte de ruído, ao passo que a regressão agregada no OSWorld-Verified (Seção 4.3) está. Sobre essa taxa de sucesso, as fontes de variância não estimadas já discutidas se acumulam. Por isso, a diferença de 3,8 pontos percentuais entre o modelo base e o treinado não pode ser isolada com segurança do ruído acumulado. A atribuição da regressão à sub-terminação, contudo, não depende dessa diferença. A análise por tarefa (Seção 4.3.1) localiza a divergência em um conjunto pequeno e identificado de tarefas, com concordância entre os modelos na ampla maioria dos casos, e por apoiar-se nesse padrão de comportamento, e não na

pontuação agregada, resiste melhor à variância de medição do que a diferença numérica isolada.

5 CONSIDERAÇÕES FINAIS

Este trabalho investigou o ajuste fino de um agente compacto de uso de computador por aprendizado por reforço sob a restrição de *hardware* comercial, com o objetivo de diagnosticar as capacidades adquiridas, as vulnerabilidades remanescentes e a dinâmica de treinamento do modelo. A avaliação recaiu sobre um ponto de verificação em estágio preliminar de convergência, correspondente ao processamento de cerca de 10% do conjunto de dados, de modo que as conclusões a seguir constituem um referencial inicial estruturado, e não a caracterização de um modelo plenamente convergido.

No *ScreenSpot-Pro*, o ajuste produziu um ganho consistente de ancoramento espacial (*grounding*), de cinco pontos percentuais na acurácia geral, concentrado na localização de elementos textuais (Seção 4.2). O aprimoramento emergiu de forma implícita, impulsionado apenas pela exigência de acerto espacial da ação prevista, sem um objetivo de otimização dedicado à detecção. O resultado indica ser viável desenvolver a precisão de ancoramento em conjunto com o aprendizado de políticas de navegação, aproveitando a base perceptiva sólida do Qwen3-VL.

No *OSWorld-Verified*, o modelo ajustado apresentou uma leve regressão na pontuação agregada em relação à rede base (Seção 4.3). A análise por tarefa e a inspeção comportamental atribuíram essa queda à sub-terminação, e não à perda de capacidade perceptiva ou de manipulação da interface (Seção 4.3.1). O agente preservou, e por vezes superou, a competência de execução das etapas intermediárias de navegação, falhando primordialmente em reconhecer a conclusão da tarefa e encerrar a trajetória. O treino preliminar demonstrou que essa política de terminação pode ser aprendida quando a ação correspondente recebe representação suficiente durante a atualização dos parâmetros (Seção 4.5). Assim, atribui-se a limitação observada à curta duração do treino e à escassez da transição terminal, em vez de uma barreira intrínseca da rede.

5.1 Dificuldades enfrentadas no desenvolvimento do estudo

A restrição ao uso de *hardware* comercial condicionou o desenho experimental como um todo e concentrou as principais dificuldades práticas do trabalho. Essa premissa motivou a adoção do regime *off-policy* sobre dados pré-amostrados, da formulação de passo único e do GSPO, que dispensa um modelo crítico e reduz o consumo de memória de vídeo (Seções 3.4.3 e 3.4.4). Ainda assim, o custo temporal permaneceu elevado. O treino preliminar, conduzido em uma única placa de consumo, levou cerca de dezesseis dias para processar aproximadamente metade do subconjunto sem ações de *grounding*, o que

corresponde a cerca de um doze avos da base completa. Diante do limite de tempo imposto pelo cronograma, recorreu-se a uma infraestrutura de servidor alugada para a configuração principal, empregada exclusivamente para ampliar o *batch* e reduzir o tempo de treino, com o número de gerações por amostra e o limite de sequência mantidos em valores compatíveis com *hardware* comercial (Seção 3.6). Por tratar-se de um recurso pago com acesso limitado, seu uso permaneceu sujeito a um orçamento restrito de tempo e de recursos, limitando o treino principal a cerca de 10% da base. Restrições de recursos e cronograma também motivaram a ausência de varreduras abrangentes de parâmetros-chave, uma vez que cada configuração adicional multiplicaria um custo já elevado.

Uma segunda ordem de dificuldade manifestou-se na estreiteza do sinal de aprendizado (Seção 4.4). Como o treinamento parte de uma base já ajustada para o seguimento de instruções, a recompensa de formato saturou nos primeiros passos, restando a acurácia como a única dimensão com margem de evolução, e ainda assim confinada a uma faixa estreita (Seção 4.4.1). A esse quadro somou-se a erosão progressiva da variância intragrupo de que o GSPO depende. À medida que as gerações encurtaram, elevou-se a fração de grupos com desvio padrão nulo, que não produzem gradiente e reduzem o sinal efetivo de treino (Seção 4.4.2).

A escassez estrutural da ação de encerramento constituiu a dificuldade mais diretamente ligada à limitação de desempenho observada. Sob a formulação de passo único, a ação `terminate` é o alvo correto em cerca de 5% dos estados do treino principal, proporção que decorre da geometria das trajetórias e induz um forte viés de continuidade (Seção 4.5). No plano da avaliação, o não-determinismo dos ambientes dinâmicos do *OSWorld-Verified* dificultou a quantificação precisa das diferenças de desempenho (Seção 4.6.3). Por fim, a extração de rótulos a partir das trajetórias orgânicas do AgentNet, densas, mas permeadas por passos redundantes ou incorretos e por tarefas abandonadas, exigiu um processo dedicado de filtragem e curadoria para preservar apenas demonstrações úteis e coerentes (Seção 3.5.1).

5.2 Alcance das conclusões e suas limitações

As conclusões deste trabalho valem dentro do escopo delimitado pelas escolhas de projeto e pelas condições de execução do experimento. As limitações reunidas a seguir demarcam até onde essas conclusões alcançam, sem colocá-las em dúvida no interior desse escopo. Elas complementam as ameaças à validade discutidas na Seção 4.6, das quais se distinguem por delimitarem a generalização dos resultados devido ao arranjo fixo experimental usado.

A formulação da recompensa e seus pesos (w_1 e w_2) entraram no experimento como fatores fixos, não variados entre as configurações (Seção 3.3), e os pesos foram

fixados empiricamente, sem um estudo de sensibilidade. Variar a formulação e mensurar a suscetibilidade aos pesos inserem-se entre os desenvolvimentos naturais do trabalho. Toda conclusão sobre o comportamento do modelo ajustado vale sob essa formulação específica, de modo que os resultados caracterizam o modelo produzido por essa recompensa, e não o espaço de modelos alcançáveis sob formulações alternativas.

O sinal de recompensa de processo valida as ações previstas em relação a uma única demonstração humana, por meio de pareamento sequencial estrito (Seção 3.5.2). Interfaces gráficas, contudo, admitem múltiplos caminhos válidos para um mesmo objetivo, como o uso de um menu ou de um atalho equivalente. O pareamento em relação a uma única trajetória penaliza rotas legítimas distintas de forma sistemática, e não aleatória, de modo que a recompensa de processo subestima a competência do modelo em uma direção enviesada. Uma consequência é que um platô na curva de acurácia não distingue entre um modelo que deixou de aprender e um que aprendeu rotas válidas porém não recompensadas. Recompensas capazes de acomodar múltiplas rotas válidas constituem uma solução potencial direta para atenuar esse viés. Nota-se que essa limitação incide sobre a interpretação da recompensa de acurácia registrada no treino, e não sobre as respostas finais, medidas de forma independente pelos *benchmarks*.

A formulação de passo único mede o acerto de cada ação condicionado a um histórico pré-registrado, e não o sucesso em execução autônoma encadeada (Seção 3.4.3). Em operação real, os erros do próprio agente compõem o contexto das decisões seguintes e se acumulam ao longo da trajetória, efeito estruturalmente ausente quando cada passo é avaliado sobre um histórico correto e fixo. Os resultados de processo caracterizam, assim, a competência por passo, enquanto a competência em execução encadeada é aferida apenas pelas respostas finais.

O escopo operacional do experimento restringe-se a uma única família de modelos, o Qwen3-VL, ao domínio exclusivamente de *desktop* e ao regime *off-policy* (Seção 3.1). As conclusões generalizam-se dentro desses limites e não se estendem, sem investigação adicional, a outras famílias de modelos, a ambientes móveis ou *web*, ou a regimes *on-policy* de interação em tempo real.

O treino principal processou apenas uma fatia do AgentNet. Nessa fatia, a ordem e o subconjunto dos dados efetivamente vistos não foram controlados (Tabela 2). A distribuição observada durante o ajuste pode não ser representativa da base completa, reforçando a leitura das métricas quantitativas como um retrato de tendências em formação, e não como valores estabilizados.

5.3 Continuidade da pesquisa e trabalhos futuros

O estágio preliminar em que este estudo se encerra delineia um percurso claro para o seu prosseguimento. Muitos dos encaminhamentos a seguir respondem diretamente às dificuldades e limitações discutidas nas seções anteriores, e visam converter o referencial empírico obtido em ganhos concretos de desempenho e de robustez. A providência mais direta consiste em estender a duração do treino para processar a totalidade do conjunto de dados, o que favorece a estabilização da política, a evolução das métricas internas de recompensa e uma exposição suficiente à ação de encerramento. A execução de estudos de ablação controlados, variando um fator por vez ao longo de múltiplas rodadas, permitiria isolar o efeito de fatores que hoje variam em conjunto, como o tamanho do *batch*, os pesos da função multicritério e a variante do espaço de ações (Tabela 3). Réplicas com sementes distintas complementariam esses estudos ao viabilizar a estimativa da variância inerente ao próprio ajuste e a construção de intervalos de confiança em torno das métricas reportadas.

A função de recompensa admite refinamentos que a tornariam mais contínua e informativa. Os pesos fixos da função multicritério poderiam ceder lugar a um esquema dinâmico. Partindo dos valores atuais, o peso atribuído ao formato seria progressivamente redistribuído em favor da recompensa de acurácia à medida que o primeiro componente satura, o que concentraria o sinal de aprendizado na dimensão que ainda dispõe de margem. A substituição do limiar binário de tolerância por retornos proporcionais à distância do alvo nas coordenadas de tela ofereceria um gradiente de otimização mais suave para as ações de ancoramento. Recompensas em nível de trajetória reforçariam o acerto temporal da ação de encerramento, atacando diretamente a sub-terminação, ao passo que recompensas capazes de aceitar múltiplas rotas válidas mitigariam o viés de penalização associado à referência de trajetória única. Para amenizar o desequilíbrio na frequência das ações, a incorporação de amostragem dinâmica ao longo das épocas fortaleceria a predição dos comandos minoritários.

Uma frente distinta decorre da própria natureza dos modelos compactos. O regime de poucos parâmetros encontra sua maior aptidão em tarefas específicas e bem delimitadas, nas quais a capacidade do modelo pode ser concentrada em uma distribuição estreita de estados e ações. O presente estudo, ao treinar e avaliar um agente genérico em múltiplos domínios de *desktop*, situa-se no extremo oposto desse espectro. Investigar o ajuste fino de CUAs compactos para domínios especializados, como uma única família de aplicações ou um fluxo de trabalho específico, constitui, portanto, uma direção promissora e alinhada às vantagens da escala compacta. Essa especialização demandaria o desenho de avaliações próprias, focadas no domínio-alvo, e poderia, adicionalmente, atenuar dificuldades observadas neste trabalho, como a sub-representação da ação de encerramento.

Ainda no plano das estratégias de especialização, uma via complementar reside

na comparação sistemática entre o aprendizado por reforço e técnicas de autodestilação como métodos primários para o refinamento de arquiteturas compactas. Contrastar as duas metodologias sob a ótica do custo computacional, da estabilidade de convergência e da resiliência do agente final ajudaria a definir a abordagem mais eficaz sob restrição estrita de *hardware* de consumo, além de fornecer parâmetros para nortear decisões de projeto nesse regime.

Em conclusão, este estudo consolidou um referencial empírico prático para a evolução de agentes de navegação locais sob restrições severas de infraestrutura e em escala preliminar. A constatação de ganhos implícitos de ancoramento e o mapeamento preciso das falhas de planejamento indicam que a estratégia de alinhamento por reforço é estruturalmente viável e amplamente aberta às otimizações aqui delineadas. A consolidação de agentes compactos capazes de operar de forma autônoma em tarefas reais permanece condicionada à superação dos desafios de representação temporal e de encadeamento de decisões identificados ao longo do trabalho, para os quais os caminhos traçados oferecem um roteiro embasado.

REFERÊNCIAS

ANTONY, J. *Design of Experiments for Engineers and Scientists*. 3. ed. Amsterdam: Elsevier, 2023. ISBN 978-0-443-15173-6.

BAI, H. et al. DigiRL: Training in-the-wild device-control agents with autonomous reinforcement learning. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2024.

BAI, S. et al. *Qwen2.5-VL Technical Report*. 2025. ArXiv preprint arXiv:2502.13923.

BAI, S. et al. *Qwen3-VL Technical Report*. 2025. ArXiv preprint arXiv:2511.21631.

Chameleon Team. Chameleon: Mixed-modal early-fusion foundation models. *arXiv preprint arXiv:2405.09818*, 2024. ArXiv:2405.09818.

CHEN, Z. et al. How far are we to GPT-4V? closing the gap to commercial multimodal models with open-source suites. *arXiv preprint arXiv:2404.16821*, 2024. ArXiv:2404.16821.

CHEN, Z. et al. InternVL: Scaling up vision foundation models and aligning for generic visual-linguistic tasks. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. [S.l.: s.n.], 2024. ArXiv:2312.14238.

CHENG, K. et al. SeeClick: Harnessing GUI grounding for advanced visual GUI agents. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. [S.l.: s.n.], 2024.

DeepSeek-AI. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. 2025. ArXiv preprint arXiv:2501.12948.

DENG, X. et al. Mind2Web: Towards a generalist agent for the web. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2023.

GAO, C. et al. UI-Venus-1.5 technical report. *arXiv preprint arXiv:2602.09082*, 2026. ArXiv:2602.09082.

GUR, I. et al. A real-world webagent with planning, long context understanding, and program synthesis. *arXiv preprint arXiv:2307.12856*, 2023. ArXiv:2307.12856; publicado na ICLR 2024.

HONG, W. et al. *CogAgent: A Visual Language Model for GUI Agents*. 2023. ArXiv preprint arXiv:2312.08914.

HONG, W. et al. CogAgent: A visual language model for gui agents. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. [S.l.: s.n.], 2024. ArXiv:2312.08914.

KAPOOR, R. et al. OmniACT: A dataset and benchmark for enabling multimodal generalist autonomous agents for desktop and web. In: *Computer Vision – ECCV 2024*. [S.l.: s.n.], 2024. ArXiv:2402.17553.

KULLBACK, S.; LEIBLER, R. A. On information and sufficiency. *The Annals of Mathematical Statistics*, v. 22, n. 1, p. 79–86, 1951.

LAI, H. et al. *ComputerRL: Scaling End-to-End Online Reinforcement Learning for Computer Use Agents*. 2025. ArXiv preprint arXiv:2508.14040.

LI, H. et al. AutoGUI: Scaling gui grounding with automatic functionality annotations from llms. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*. [S.l.: s.n.], 2025. ArXiv:2502.01977.

LI, J. et al. *BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation*. 2022. ArXiv preprint arXiv:2201.12086.

LI, K. et al. *ScreenSpot-Pro: GUI Grounding for Professional High-Resolution Computer Use*. 2025. ArXiv preprint arXiv:2504.07981.

LI, W. et al. On the effects of data scale on computer control agents. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2024. ArXiv:2406.03679.

LIU, H. et al. *Visual Instruction Tuning*. 2023. ArXiv preprint arXiv:2304.08485.

LIU, H. et al. Visual instruction tuning. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2023. ArXiv:2304.08485.

LIU, Y. et al. *InfiGUI-R1: Advancing Multimodal GUI Agents from Reactive Actors to Deliberative Reasoners*. 2025. ArXiv preprint arXiv:2504.14239.

LU, Q. et al. GUI Odyssey: A comprehensive dataset for cross-app gui navigation on mobile devices. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. [S.l.: s.n.], 2025. ArXiv:2406.08451.

LU, Z. et al. *UI-R1: Enhancing Efficient Action Prediction of GUI Agents by Reinforcement Learning*. 2025. ArXiv preprint arXiv:2503.21620.

LUO, R. et al. *GUI-R1: A Generalist R1-Style Vision-Language Action Model for GUI Agents*. 2025. ArXiv preprint arXiv:2504.10458.

MENG, L. et al. *DeepStack: Deeply Stacking Visual Tokens is Surprisingly Simple and Effective for LMMs*. 2024. ArXiv preprint arXiv:2406.04334.

OpenAI. *ChatML: Chat Markup Language*. 2023. Acesso em: 28 jun. 2026. Disponível em: <https://github.com/openai/openai-python/blob/main/chatml-.md>https://github.com/openai/openai-python/blob/main/chatml.md.

OUYANG, L. et al. Training language models to follow instructions with human feedback. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2022. ArXiv:2203.02155.

QIN, Y. et al. *UI-TARS: Pioneering Automated GUI Interaction with Native Agents*. 2025. ArXiv preprint arXiv:2501.12326.

RADFORD, A. et al. *Improving Language Understanding by Generative Pre-Training*. [S.l.], 2018.

RADFORD, A. et al. *Learning Transferable Visual Models From Natural Language Supervision*. 2021. ArXiv preprint arXiv:2103.00020.

RADFORD, A. et al. *Language Models are Unsupervised Multitask Learners*. [S.l.], 2019.

SCHULMAN, J. et al. *Trust Region Policy Optimization*. 2015. ArXiv preprint arXiv:1502.05477.

SCHULMAN, J. et al. *Proximal Policy Optimization Algorithms*. 2017. ArXiv preprint arXiv:1707.06347.

SHAO, Z. et al. *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*. 2024. ArXiv preprint arXiv:2402.03300.

SUN, Q. et al. OS-Genesis: Automating gui agent trajectory construction via reverse task synthesis. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*. [S.l.: s.n.], 2025. ArXiv:2412.19723.

VASWANI, A. et al. *Attention Is All You Need*. 2017. ArXiv preprint arXiv:1706.03762.

WANG, H. et al. *UI-TARS-2 Technical Report: Advancing GUI Agent with Multi-Turn Reinforcement Learning*. 2025. ArXiv preprint arXiv:2509.02544.

WANG, P. et al. *Qwen2-VL: Enhancing Vision-Language Model's Perception of the World at Any Resolution*. 2024. ArXiv preprint arXiv:2409.12191.

WANG, X. et al. *OpenCUA: Open Foundations for Computer-Use Agents*. 2025. ArXiv preprint arXiv:2508.09123.

WILLIAMS, R. J. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, v. 8, n. 3-4, p. 229–256, 1992.

WU, Z. et al. OS-ATLAS: A foundation action model for generalist gui agents. In: *The Thirteenth International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2025. ArXiv:2410.23218.

XIE, T. et al. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2024.

XU, Y. et al. *Aguvis: Unified Pure Vision Agents for Autonomous GUI Interaction*. 2024. ArXiv preprint arXiv:2412.04454.

YAO, Y. et al. MiniCPM-V: A gpt-4v level mllm on your phone. *arXiv preprint arXiv:2408.01800*, 2024. ArXiv:2408.01800.

ZHANG, Z. et al. AgentCPM-GUI: Building mobile-use agents with reinforcement fine-tuning. *arXiv preprint arXiv:2506.01391*, 2025. ArXiv:2506.01391.

ZHENG, B. et al. GPT-4V(ision) is a generalist web agent, if grounded. In: *Proceedings of the 41st International Conference on Machine Learning (ICML)*. [S.l.: s.n.], 2024. ArXiv:2401.01614.

ZHENG, C. et al. *Group Sequence Policy Optimization*. 2025. ArXiv preprint arXiv:2507.18071.

ZHOU, S. et al. WebArena: A realistic web environment for building autonomous agents. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2024.

ZHOU, Y. et al. *GUI-G1: Understanding R1-Zero-Like Training for Visual Grounding in GUI Agents*. 2025. ArXiv preprint arXiv:2505.15810.

6 APÊNDICE

6.1 *Prompt* do sistema do agente

Reproduz-se a seguir, na íntegra, o *prompt* de sistema fornecido ao modelo, conforme descrito na Seção 3.4.6. O texto define o comportamento esperado do agente, o formato do histórico de navegação recebido, o padrão de saída exigido, o espaço de ações disponível e exemplos de resposta. É apresentado em seu idioma original (inglês), tal como utilizado nos experimentos.

```
You are a desktop GUI agent. Given a screenshot of the current screen and a task
    instruction, decide the next action(s) to take.

## Conversation history format
Earlier turns are a log of what has already been done. They appear in three tiers,
    from newest to oldest:
- Most recent prior step: the screenshot is shown directly as an image, followed by
    a text <observation> of that screen and a bare action list.
- The two steps before that: a text <observation> describing the screen state at
    that point, followed by a bare action list. No screenshot.
- All older steps: only a bare action list - no observation and no screenshot. Treat
     these as a trail of actions already attempted, not as a reconstruction of past
    screens.
YOUR own reply must NOT use any of these compact formats; it must follow the full
    format below.

## Output format - your response must follow this exactly
Begin with <observation>, then <reason>, then <answer>, and STOP. No preamble. No
    text after </answer>.

Skeleton:
<observation>OBSERVATION</observation> <reason>REASONING</reason> <answer>[{'
    description': 'DESCRIPTION', 'action': 'ACTION_CALL'}]</answer>

Where:
- OBSERVATION: a concise description of the current screen state - what application
    is open, what is visible, where the cursor/focus appears to be, any notable UI
    elements. Two to four sentences. This is your grounding step before reasoning.
- REASONING: a structured analysis (see 'How to reason' below). Keep it focused and
    write in the first person.
- The <answer> body is a Python list literal of dicts. Each dict MUST have BOTH keys
    :
    'description' - a short natural-language summary of what this action does and
```

```
    why. Refer to the target by its visible name, label, or distinguishing features
    (shape, color, position) - NOT by coordinates. For window controls, identify
    them correctly: minimize is '-', maximize is '□', close is 'X'. If a UI element'
    s name is in a non-English language, keep the original language.
    'action'      - a single action call string from the table below, in the exact
    syntax shown.
- You MAY emit multiple action dicts in the list if several actions should fire in
    sequence at this step (e.g. write then press enter, or consolidated repeated
    keypresses). Most steps are a single action - don't pad. Prefer consolidating
    repetitive keypresses using the 'presses' kwarg (e.g. press('backspace', presses
    =5)) over emitting five separate dicts.
- Quoting: write each 'action' value as a DOUBLE-quoted Python string so any single
    quotes inside (e.g. press('enter')) are valid. Example: 'action': "press('enter
    ')". Never write 'action': 'press('enter')' - that is a syntax error.
- Coordinates x and y are NORMALIZED FLOATS in [0.0, 1.0] - never raw pixel values.
    Always include a leading 0 (write 0.5, not .5).
- Use ONLY the actions in the table below, with their exact argument syntax. Do NOT
    invent actions like type(...), execute(...), or bare shell commands.

## How to reason - what goes inside <reason>...</reason>
Structure your reasoning around two phases. You do not need literal headers, but
    every reply should cover both:

1. Progress assessment - look backward:
   - What parts of the overall task are already done, based on the prior action log
    and the current screenshot?
   - Did the previous action produce the expected result? Look for unexpected
    dialogs, error states, unchanged screens, or focus loss.
   - If the previous action appears to have failed or led somewhere wrong, name the
    recovery step (dismiss the dialog, undo, retry with a different target, etc.)
    instead of plowing ahead.

2. Next-action analysis - look forward:
   - Briefly consider the plausible next actions given the current screen state.
   - Pick the most logical one and say why it beats the alternatives (e.g. keyboard
    shortcut vs. menu navigation, direct click vs. scrolling first).
   - Anticipate what the screen should look like after the action - this becomes the
     check on the next turn.

Special case - text input actions:
   - Note where the cursor currently is and whether the intended field is actually
    focused (don't assume).
   - State the expected final text in the field after typing.
   - Consolidate repeated keypresses with the 'presses' kwarg rather than repeating
    dicts.

Special case - terminating:
```

```
    - Only call terminate(status='success') when the screenshot itself confirms the
    task's end state. Do not terminate optimistically.
    - Call terminate(status='failure') when you have exhausted reasonable approaches
    or the task is genuinely blocked, and explain why in the reasoning.

## Available actions
Pointer actions - kwargs x, y as floats in [0.0, 1.0]:
  click(x=0.5, y=0.3)                       left-click at the given point
  rightClick(x=0.5, y=0.3)                  right-click
  doubleClick(x=0.5, y=0.3)                 double-click
  tripleClick(x=0.5, y=0.3)                 triple-click (e.g. select a line of text
    )
  middleClick(x=0.5, y=0.3)                 middle-click
  moveTo(x=0.5, y=0.3)                      move the cursor without clicking
  dragTo(x=0.5, y=0.3, button='left')       drag to the point; button is one of '
    left' | 'right' | 'middle'

Scrolling - a single positional INTEGER (no decimals, no kwargs); positive = down/
    right, negative = up/left:
  scroll(-3)                                vertical scroll
  hscroll(2)                                horizontal scroll

Keyboard:
  write(message='hello world')              type literal text into the focused field
     (kwarg 'message'; do NOT use type())
  press('enter')                            press a single key by name
  press('a', presses=3)                     press a key multiple times (optional
    kwarg) - prefer this over repeating dicts
  press(['tab', 'tab'])                     press a sequence of keys, in order
  hotkey(['ctrl', 'c'])                     hold a chord of keys simultaneously

Navigation / control:
  wait()                                    pause briefly for the UI to update
  terminate(status='success')               end the task - use when complete
  terminate(status='failure')               end the task - use when it cannot be
    completed

## Worked examples (your reply should look like these - every reply, even if the
    history has bare actions)
Example A - single click, with progress + next-action reasoning:
<observation>The screen shows a text editor application with a menu bar at the top.
    The File, Edit, View, and Help menus are visible. The document area below is
    empty and no menus are currently open. The cursor appears to be in the main
    document body.</observation> <reason>The history shows I haven't opened any
    menus yet, and the current screenshot confirms I'm at the application's main
    window. The File menu sits in the top-left of the toolbar, and I need it to
    reach 'Open Recent'. Alternatives - a keyboard shortcut like Ctrl+O - would skip
```

```
     the recent-files list, so the menu is the right path. After clicking I expect
    the File dropdown to appear, exposing 'Open Recent' as a submenu.</reason> <
    answer>[{'description': 'Open the File menu in the top-left of the menu bar', '
    action': "click(x=0.04, y=0.03)"}]</answer>

Example B - keyboard shortcut, recovering from a previous misstep:
<observation>The screen shows a document editor with text content visible. The
    cursor is blinking inside the document body, indicating the editor has focus.
    The title bar shows the filename without a dirty indicator, but I have not yet
    saved the changes made in this session.</observation> <reason>The previous
    action clicked into the document body, which means the editor is now focused -
    good, that's the prerequisite for a save shortcut. The task is to save the file,
     and I haven't done that yet. Ctrl+S is the standard save shortcut and avoids
    menu navigation entirely. After pressing it I expect either a silent save (title
     bar dirty marker disappears) or a Save As dialog if the file is untitled.</
    reason> <answer>[{'description': 'Save the document with the Ctrl+S shortcut', '
    action': "hotkey(['ctrl', 's'])"}]</answer>
```